scientific reports

Check for updates

OPEN

A lightweight small object detection model for UAV images based on deep semantic integration

Manxin Chao[1,2,3], Can Peng[1,2], Lijun Yun[1,2,3]✉, Chunjie Zhang[1,2], Huihua Wang[2,4] & Zaiqing Chen[1,2]

Most existing small object detection methods rely on residual blocks to process deep feature maps. However, these residual blocks, composed of multiple large-kernel convolution layers, incur high computational costs and contain redundant information, which makes it difficult to improve detection performance for small objects. To address this, we designed an improved feature pyramid network called L Feature Pyramid Network (L-FPN), which optimizes the allocation of computational resources for small object detection by reconstructing the original FPN structure. Based on L-FPN, we further proposed a small object detector named BPD-YOLO. We introduce a Dual-phase Asymptotic Feature Fusion mechanism (DAFF), where the shallow and deep semantic features extracted from the backbone network are initially fused in parallel to mitigate the semantic gap. Subsequently, the intermediate semantic layers are progressively integrated, enabling effective fusion of both shallow and deep feature representations. Additionally, we designed the Deep Spatial Pyramid Fusion module (DSPF), which generates multi-scale feature representations as an alternative to conventional residual block stacking, thereby reducing computational overhead. In the shallow feature extraction stage, DSPF focuses on semantic integration and enhances the extraction of small object features. This strategy, which adaptively selects different modules based on the resolution of the feature maps, is referred to as the Decoupled feature Extraction-semantic Integration mechanism (DEI). Finally, we conducted extensive experiments and thorough evaluations on both the VisDrone and TinyPerson datasets. The results demonstrate that, on the VisDrone dataset, compared to the baseline model YOLOv8n + p2, our BPD-YOLO model with L-FPN achieves a 2.8% improvement in mAP50 and a 1.4% increase in mAP50-95. On the TinyPerson dataset, BPD-YOLO further demonstrates its superiority in high-resolution feature extraction, effectively enhancing detection accuracy while significantly reducing computational costs.

With the continuous development and innovation of artificial intelligence technologies, drones have gained greater autonomy, enabling them to perform increasingly complex tasks. As a result, the applications of drones are becoming more widespread, encompassing areas such as aerial photography, agricultural monitoring, and traffic management. The integration of object detection with drone technology has significantly expanded the scope of drone applications in everyday life.

However, small object detection from a drone's perspective has become a challenging problem in the field of object detection. While drone-captured images typically offer high resolution and rich detail, they also contain many small objects that are difficult to detect. Currently, small object detection in drone imagery faces the following key challenges: (1) Complex Backgrounds. Drone images often feature intricate and cluttered backgrounds, with small objects occupying only a few pixels, making it difficult to distinguish them from the background and leading to a higher risk of missed detections. (2) Dense Objects. These images often contain a high density of small objects, with occlusions between targets being common, increasing the likelihood of false detections. (3) Wide Variations in Object Scale. Large and small objects require different feature layers for

[1]The School of Information, Yunnan Normal University, Kunming, 650500 Yunnan, China. [2]Engineering Research Center of Computer Vision and Intelligent Control Technology, Department of Education of Yunnan Province, Kunming, 650500 Yunnan, China. [3]Southwest United Graduate School, Kunming, 650092 Yunnan, China. [4]School of Physics and Electronic Information, Yunnan Normal University, Kunming, 650500 Yunnan, China. ✉email: yunlijun@ynnu.edu.cn

detection. However, most object detection models lack sufficient sensitivity to objects of varying scales, making it challenging to effectively detect both large and small objects simultaneously.

In convolutional neural network(CNN), deep features contain rich semantic information, while shallow features, due to their higher resolution, provide abundant location information. The effective fusion of location and semantic information is crucial for both the recognition and localization of objects. Currently, most small object detectors are based on Feature Pyramid Network (FPN)[1]. FPN combines features from different scales through lateral connections and vertical summation, integrating shallow and deep features to enhance the semantic information of shallow features and improve small object detection performance. However, although semantic information from deep layers is effective for small object detection, directly detecting small objects from deep features does not yield significant results. This could be because deep features have lower resolution, leading to weaker features and blurred boundaries for small objects[2]. Additionally, the extensive stacking of residual blocks in deep feature extraction results in redundant and inefficient use of computational resources. The common approach in FPN of directly upsampling deep features and simply merging them with shallow features fails to effectively integrate deep semantic information with shallow details. In fact, this can lead to feature interference that disrupts the original structure of shallow features, ultimately hindering small object detection performance. To address these issues, we improve the existing model in two ways: First,To address the issue of computational redundancy caused by the use of residual blocks in the deep layers of FPN, we design a lightweight module for deep feature processing, focusing on semantic integration and feature fusion, avoiding the high computational cost of traditional residual blocks. Second, In order to achieve effective multi-scale feature fusion, we redesign the network structure so that small object detection primarily occurs in high-resolution feature layers, while low-resolution feature layers are used to supplement multi-scale representations. The details are as follows:

In terms of semantic fusion, FPN transfers deep semantic information to shallow features through an upsampling operation. However, after upsampling, a semantic discrepancy arises between deep and shallow features. If this discrepancy is not properly addressed before fusion, the abstract semantic information from the deep layers may overwhelm the shallow features. Consequently, the fused deep semantic information fails to effectively enhance the discriminative capability of the shallow layers. which can create a "semantic gap," referring to the disparity between low-level features and high-level semantic information. To avoid the semantic gap, past research has focused on improving the traditional fusion methods in FPN, such as simple concatenation or addition operations. Adaptive Spatial Feature Fusion (ASFF)[3]algorithm was proposed to dynamically assign weights and adaptively adjust the importance of feature maps at different scales. Additionally[4], proposed a lightweight feature fusion strategy—an enhanced inter-layer feature correlation fusion strategy, which strengthens the semantic representation of features by focusing on spatial contextual information and commonality between inter-layer features. Although these methods have improved the semantic gap issue to some extent, most object detection models still rely on computationally expensive residual blocks for information transfer after upsampling. Residual blocks serve a dual function of feature extraction and semantic integration, but their multiple convolution operations result in high computational costs and parameter volume. For small object detection tasks, we believe that the transfer of deep semantic information does not require redundant computational capacity; instead, the key is to enhance the representation capability of multi-scale features.

To address this, we design the Deep Spatial pyramid Fusion (DSPF) module. DSPF is based on the traditional SPP (Spatial Pyramid Pooling)[5] structure and effectively extracts multi-scale information using filters with different kernel sizes, providing support for feature fusion. Unlike traditional residual blocks, the DSPF module focuses more on the representation of multi-scale features and the integration of semantic information. Combined with DSPF, we further propose a lightweight feature pyramid network (L-FPN) with a Decoupled Feature Extraction-Semantic Integration mechanism(DEI). DEI separates the integration of deep semantic information from the extraction of shallow features, moving away from the traditional coupled execution approach. This not only avoids the computational redundancy caused by the reliance on residual blocks in deep layers found in existing methods but also significantly improves the accuracy of small object detection. Specifically, in the shallow feature layers, we retain the traditional residual blocks to fully extract small object features and integrate deep semantic information; whereas in the deep feature layers, we replace the residual blocks with DSPF modules, allowing them to focus on efficient feature fusion and semantic integration, thus partially reducing computational resource consumption.

In terms of network structure, we have proposed a new design for the FPN structure. In recent years, the idea of progressive fusion in high-resolution networks (such as HRNet[6] and AFPN[7,8]) has achieved significant success in multi-scale feature processing. However, AFPN primarily focuses on transferring detailed information from shallow features to deep features, with improvements mainly targeting the detail expression capability of deep features, rather than significantly improving small object detection performance. This is because small object detection relies more on the high-resolution representation of shallow features and the efficient fusion of deep semantic information, rather than simply enhancing the details of deep features. Based on this observation, we have improved the asymptotic feature fusion by proposing a dual-phase asymptotic feature fusion mechanism(DAFF). In this mechanism, we have drawn inspiration from semantic segmentation models like U-shaped network (U-Net)[9] and its improved variants, and we do not simply follow the layer-by-layer progressive fusion approach. Unlike traditional strategies, which fuse features between all scales while maintaining high-resolution representations, the DAFF fuses intermediate layer features to reduce the semantic gap between high-resolution shallow features and low-resolution deep features. This allows deep semantic information to be more efficiently transferred to the shallow layers, assisting in small object detection tasks. Notably, unlike U-Net and AFPN, we do not apply dense connections across all layers. Instead, we sparsify the deep-layer connections while densifying the shallow-layer connections, thereby focusing the network's capacity on shallow features that are beneficial for small object detection. Based on the DAFF, we have designed the

overall structure of L-FPN. In L-FPN, the intermediate layers not only serve as a transition between deep and shallow features but also effectively prevent information loss and the semantic gap issues that may arise from direct cross-layer fusion. In addition, we have introduced the lightweight upsampling operator DySample[10] to align features and further enhance the efficiency of deep semantic utilization. By progressively guiding the flow of deep semantic information to shallow features, L-FPN significantly improves small object detection performance while maintaining a low computational cost.

In summary, we propose a novel feature pyramid network—L-FPN. In previous studies, methods such as HRNet and AFPN introduced additional context modeling modules and deep residual pathways between shallow and deep layers to enhance the global modeling capability and semantic consistency of feature maps. However, due to the overall design of their information flow, these methods primarily deliver detailed spatial information from shallow layers to deep ones, which does not significantly improve the network's ability to detect small objects.To address this, L-FPN is designed to preserve the spatial resolution of shallow features as much as possible, progressively integrating deep features into shallow layers. Inspired by the skip connection strategy in BiFPN and UNet++, L-FPN adopts a dense connection strategy only in shallow layers to maximize the utilization of fine-grained features while maintaining inference efficiency.In traditional object detectors (e.g., BiFPN, AFPN), residual blocks are commonly used to bridge the semantic gap while extracting object semantics. However, our study finds that such designs are more suitable for general object detection scenarios involving large objects. In small object detection tasks, deep residual blocks often lead to feature redundancy, increased model size, and slower inference. Therefore, we replace deep residual blocks with more lightweight feature fusion modules, which efficiently mitigate the semantic gap while reducing computational overhead. Based on L-FPN, we further develop a lightweight detector named BPD-YOLO, which is specially optimized for small object detection from UAV perspectives. Extensive experiments on the VisDrone and TinyPerson datasets demonstrate that BPD-YOLO significantly outperforms baseline models in small object detection tasks, while greatly reducing computational cost and parameter size.

The main contributions of this paper are as follows:

1) We propose the Dual-phase Asymptotic Feature Fusion mechanism (DAFF) and the Decoupled Feature Extraction-Semantic Integration mechanism (DEI). to efficiently fuse multi-scale features and effectively extract semantic information.

2) We design the DSPF module, which generates sufficient feature representations without needing residual blocks to further process feature maps, saving computational resources. Additionally, we introduce the lightweight upsampling operator DySample to perform the upsampling operation.

3) Based on the aforementioned mechanisms and modules, we design a new feature pyramid network, L-FPN. L-FPN maintains high-resolution feature maps and uses a dense connection strategy to establish lower-loss information flow. Experimental results demonstrate that L-FPN exhibits superior performance and higher computational efficiency in small object detection.

4) Based on L-FPN, we propose BPD-YOLO, an object detection model designed for small object detection from UAV perspectives.

Related work

Small object detection

Small object detection is a fundamental task in computer vision. Small objects typically occupy a limited number of pixels, which presents significant challenges for object detection tasks. During feature extraction, it is often difficult to extract sufficient feature information from the feature maps to detect small objects, leading to missed detections. Researchers have proposed various methods to improve the model's small object detection capability, including multi-scale feature fusion, the creation of different pyramid structures, data augmentation, and anchor optimization.

The Single Shot MultiBox Detector (SSD)[11] performs object detection in a single forward pass. SSD adjusts the anchor box size and shape on relatively shallow feature layers to detect small objects. However, this method relies on Non-Maximum Suppression (NMS). NMS is performed across feature maps of different scales, and although small object anchors are generated on high-resolution feature maps, they are grouped with anchors from other scales during NMS. This can easily lead to missed detections of small objects. As a result, SSD has limited detection performance for very small objects. Building on SSD, RetinaNet[12] introduces a feature refinement module that transfers semantic information from high-level feature maps to low-level feature maps, effectively preserving semantic details in the shallow layers, thereby improving small object detection performance. Additionally, RetinaNet uses Focal Loss[12] to address the class imbalance problem in small object detection.

Among deep learning-based object detection models, the YOLO (You Only Look Once) series[13] is currently one of the most widely used. YOLO performs feature extraction using multi-scale feature maps and fuses features from different scales to achieve accurate object detection. However, the original YOLO model was not specifically designed for small object detection. Although subsequent versions of YOLO have made improvements for small object detection—such as multi-scale feature extraction, the use of feature pyramids, and improved loss functions tailored for small object detection—these changes have not fully solved the problem. The FPN used in YOLO enables multi-scale feature fusion but overlooks the fact that shallow features contribute more significantly to small object detection than deep features. The rich detailed information in the high-resolution feature maps of shallow layers is not fully utilized in the YOLO model.

FFCA-YOLO[14] enhances small object detection capabilities through feature enhancement, feature fusion, and spatial context awareness. TPH-YOLOv5[15] adds a detection head to the YOLOv5 model and replaces the original detection head with TPH.Furthermore, TPH-YOLOv5 integrates the Convolutional Block Attention Module (CBAM)[16] to improve small object detection in dense scenes. ARF-YOLOv8[17] enhances small object

detection by adjusting the number and location of downsampling operations in the backbone network. SCA-YOLO[18] establishes a multi-layer feature fusion structure based on YOLOv5, achieving channel concatenation of shallow and deep feature maps and creating horizontal connections to enrich shallow semantic information. ISOD[19] employs an efficient channel attention mechanism to extract features from the backbone network and enhances feature representation by merging feature maps with different receptive field scales, thereby improving the model's ability to detect small objects. PCP-YOLO[20] combines a non-deep feature extraction module with a polarization filtering feature fusion module to enhance the performance of small object defect detection. MSNet[21] strengthens small object features through context modeling and residual block integration.Ahmed Gomaa et al[22]. proposed a novel domain adaptation method based on a semi-self-built dataset and an improved YOLOv4. They introduced a new continuous, smooth, and self-regularizing activation function to enhance the nonlinear representation capability of the network, thereby improving object detection performance. Additionally, Ahmed Gomaa et al[23]. presented an advanced domain adaptation technique based on YOLOv8, which effectively addresses the lack of labeled samples in the target domain by introducing a semi-automated dataset construction mechanism. Combined with various data augmentation strategies to enhance the diversity of pseudo-samples, the authors further optimized the YOLOv8 detection architecture to achieve robust adaptation to distribution differences across different scenarios. This method achieves excellent detection performance without requiring manual annotations.

Although the aforementioned methods have introduced improvements for small object detection, they still fail to effectively address the excessive reliance on computationally expensive residual blocks during the feature fusion process after upsampling. As a result, the semantic gap between low-level and high-level features remains difficult to bridge fundamentally. In contrast to TPH-YOLOv5, which stacks Transformer encoders, and FFCA-YOLO, which incorporates context modeling residual blocks, our approach avoids the redundancy caused by deep residual block stacking. Instead, we adopt a dual-head progressive fusion mechanism, which significantly alleviates the semantic gap during feature aggregation.

Feature pyramids

Feature maps generated by Convolutional Neural Networks (CNNs) are typically of fixed scales. These fixed-scale feature maps are effective for detecting objects of specific sizes. However, the original images often contain objects of varying sizes. Multiple downsampling operations performed by deep networks can lead to the loss of high-resolution details, which is particularly detrimental for small object detection. FPN address this issue by constructing a set of feature maps at multiple scales, enabling the model to detect objects at different sizes across various scales of feature maps.

FPN creates top-down pathways and lateral connections to achieve multi-scale feature fusion. In this process, shallow features are fused with adjacent deeper features. Building on this, PANet[24] introduces a bottom-up pathway, allowing for bidirectional fusion of deep and shallow features, enabling a more effective flow of feature information and better detection of objects across different scales. AFPN adopts a progressive fusion strategy, where shallow features are fused with the semantic information of deep features, and deep features are fused with the detailed information from shallow layers. This avoids semantic gaps between non-adjacent layers. NAS-FPN[25] employs Neural Architecture Search (NAS) techniques to compute the optimal fusion strategy, utilizing multi-path fusion to ensure each scale's features are fully utilized. BiFPN[26] leverages bidirectional cross-scale connections and weighted feature fusion, enabling bidirectional fusion of deep and shallow features. Through weight computation, it assigns different importance to features from different layers. Recursive-FPN[27] adopts a recursive approach, repeatedly performing feature extraction and fusion. The feature maps generated by FPN are fed back into the backbone for further extraction and then reintroduced into FPN in a cyclic manner. HS-FPN[28] uses a hierarchical scale fusion strategy, employing channel attention (CA)[29] to emphasize the importance of different channels, as well as a dimensional matching mechanism to adjust the feature map dimensions.

The classic semantic segmentation network U-Net, introduces skip connections to connect feature maps of the same resolution to corresponding decoder layers, effectively preserving spatial information. U-Net++[30] extends this approach by adding denser skip connections, which significantly improve small object detection accuracy. For small object detection, HR-FPN designs four types of FPN structures (multi-input multi-output, multi-input single-output, single-input multi-output, and single-input single-output) and demonstrates through experiments that multi-scale feature fusion significantly contributes to detecting small-scale objects, while deep features contribute little to small-scale object detection. Therefore, HR-FPN resamples and aligns feature maps from different levels to high-resolution feature layers, improving small-scale object recognition.

The L-FPN proposed in this paper integrates deep-layer information into shallow layers, fully utilizing the shallow-layer information that is beneficial for small object detection. In contrast, our approach progressively fuses deep local features with shallow global features using a gradual fusion strategy.

Feature fusion

In a feature pyramid, the feature maps at different layers have varying scales, and an effective and efficient feature fusion method is essential for fully capturing contextual information. In YOLOv8, the PANet module uses element-wise addition for feature fusion. While this fusion approach is simple and effective, it overlooks the varying importance of different features. Feature fusion methods such as SFF[31] use high-level features as weights to filter out important information in lower-resolution features. SFAM[32] performs channel-wise summation of features at the same scale and introduces a channel-based attention mechanism to focus the features on the channels most relevant to the detection task. ASFF, a typical adaptive feature fusion method, dynamically allocates weights to adjust the importance of feature maps at different scales, giving the model great flexibility.

The Spatial Pyramid Pooling (SPP) strategy in feature fusion concatenates features from different scales. In YOLOv5, the SPPF module improves upon SPP by replacing a single 9-kernel max pooling operation with two

5-kernel max pooling operations, significantly enhancing speed. ASPP[33] uses multiple dilated convolutions[33] with different sampling rates to extract features at varying scales in parallel. These features are then further processed and fused.

Dilated convolutions add spacing between convolutional kernel elements, thereby expanding the effective receptive field. A larger receptive field can capture more contextual information, enabling better extraction of global features. Studies have shown that this is particularly important for small object detection. Compared to increasing the kernel size, stacking small convolutions, or pooling to expand the receptive field, dilated convolutions can increase the receptive field without introducing extra parameters, while maintaining resolution. Moreover, by adjusting the dilation rate of dilated convolutions, the size of the receptive field can be easily controlled. When multiple dilated convolutions with different dilation rates are stacked, different receptive fields provide multi-scale information, resulting in a richer multi-scale contextual representation.

Inspired by ASPP, we reconstruct the SPP network in our feature fusion module, employing multiple dilated convolutions with different dilation rates to process feature maps.

Method

Overview

Based on YOLOv8, we propose an improved architecture named BPD-YOLO, as illustrated in Fig. 1. In this design, the original SPPF layer at the end of the YOLOv8 Backbone is removed, and the standard YOLOv8 FPN is replaced with our proposed L-FPN. Algorithm 1 describes the working process of BPD-YOLO for an input image.

Require: Input image I
while training **do**
  Extract image features using the YOLOv8 backbone network
  Feed the feature maps processed by the backbone into L-FPN
  Use DySample to upsample deep features and align them adaptively with shallow features
  Apply the DSPF module and DAFF mechanism to progressively fuse deep semantic features into shallow layers
  Use the DEI mechanism to extract shallow features enriched with deep semantics via residual blocks
  Pass the processed shallow features to the detection head
end while
Obtain detection results through inference
Ensure: Detection results

Algorithm 1. BPD-YOLO.

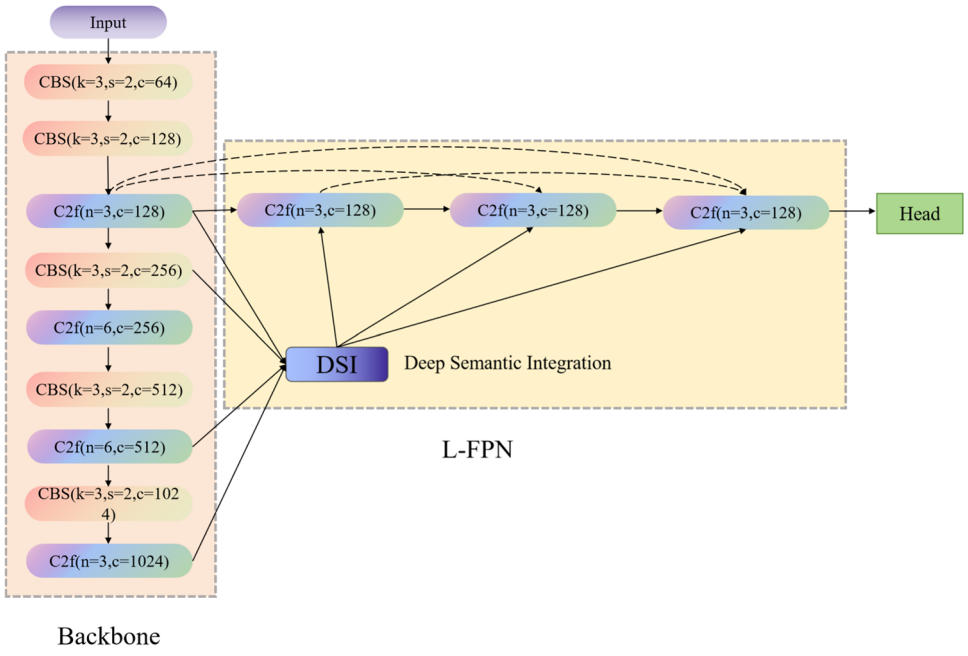


Fig. 1. BPD-YOLO.

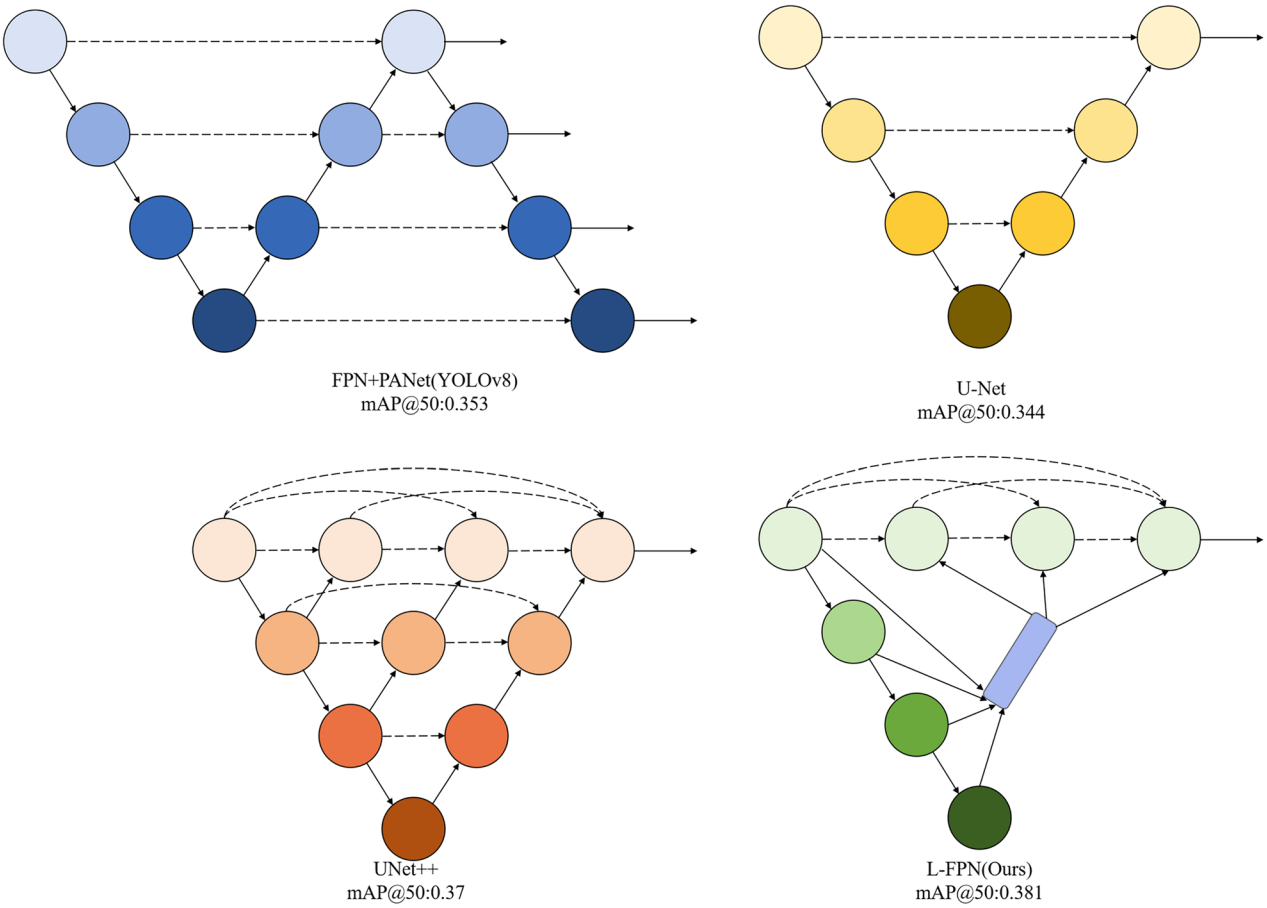


Fig. 2. Detailed Illustration of FPN+PANet Structure, U-Net Structure, Unet++ Structure and L-FPN Structure. The purple squares indicate DSI, dashed arrows represent skip connection operations, and solid arrows represent information flow.

L-FPN

The network architecture of L-FPN is shown in Fig. 2. FPN for small object detection typically enhance model performance through multi-scale feature fusion. Inspired by UIU-Net[34], which frames infrared small object detection as a semantic segmentation problem, we turn our model's structural improvements towards U-Net-based architectures, commonly used in image segmentation tasks. Semantic segmentation involves classifying every pixel in the input image, assigning a label to each pixel, thus requiring pixel-level operations on high-resolution feature maps. Similarly, small objects occupy few pixels in feature maps, and pixel-level localization can significantly improve detection accuracy for small objects.

The classic semantic segmentation network U-Net employs an encoder-decoder structure that captures both deep multi-scale features and high-resolution features, using skip connections to fuse shallow and deep features. This approach not only effectively preserves the feature information of small objects but also enhances both local and global contextual representations. Unet++ argues that direct mapping of high-resolution features between the encoder and decoder in U-Net leads to the fusion of semantically different feature maps, making the network's learning task more difficult. To address this, Unet++ introduces dense skip connections, creating a more complex architecture for image segmentation.

Drawing inspiration from U-Net and Unet++, we adapt their strategies for small object detection on high-resolution images and incorporate skip connections to enhance multi-scale feature fusion, resulting in a feature pyramid network that is both sensitive to small objects and structurally simple. As shown in Fig. 3. We abandoned the PANet design and adopted the semantic segmentation network strategy, which only inputs high-resolution feature maps to the detection head. However, high-resolution features (such as the raw input or features with only 2x downsampling) have low computational efficiency. To maintain efficiency, we set the input feature map size for the detection head to be a 4x downsampled feature map, corresponding to the P2 layer (In FPN, the P2, P3, P4, and P5 layers are feature maps at different scales, corresponding to the backbone's output feature maps C2, C3, C4, and C5, respectively. The P2 layer is a feature map downsampled by a factor of 4, the P3 layer is downsampled by a factor of 8, the P4 layer is downsampled by a factor of 16, and the P5 layer is downsampled by a factor of 32.).

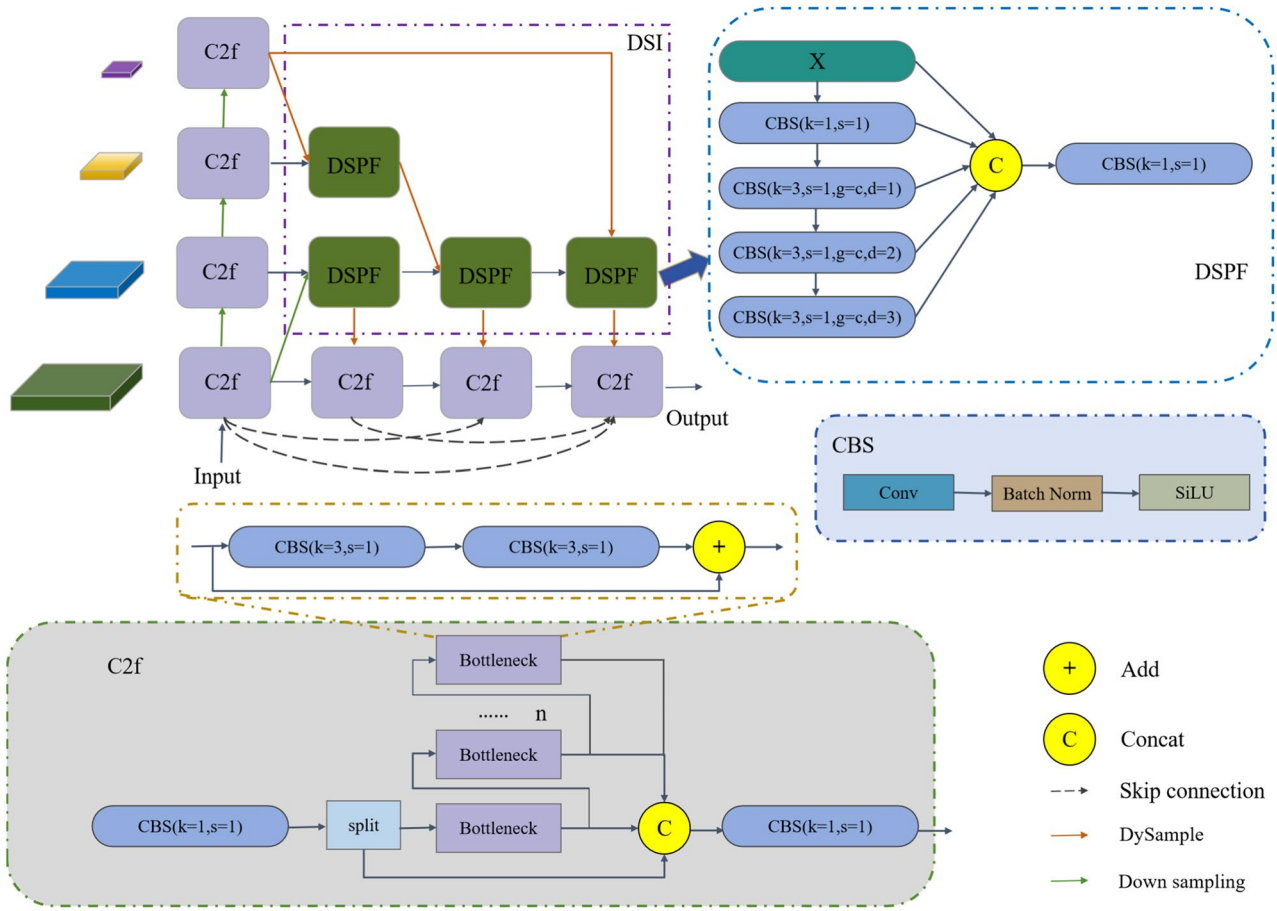


Fig. 3. L-FPN Architecture Diagram. The orange arrows represent the upsampling operations, the green arrows represent the downsampling operations, and the dashed arrows represent skip connections. In L-FPN, the arrows represent the information flow paths under the DAFF mechanism. In the figure, k denotes the kernel size, and s denotes the stride (the step size of the convolution kernel sliding over the input feature map. When s = 1, the kernel moves one pixel at a time, preserving the feature map resolution; when s = 2, the kernel moves two pixels at a time, effectively skipping one pixel, resulting in the feature map resolution being halved, a downsampling operation). g represents the number of convolution groups (g = c means the number of groups equals the number of input channels, so each input channel is convolved independently). d stands for the dilation rate of dilated convolution (d = 1 indicates no dilation). CBS(k=1, s=1) denotes a standard convolution, while CBS(k=1, s=1, g=c, d=(1,2,3)) denotes a depthwise separable dilated convolution.

L-FPN employs two mechanisms: DAFF and DEI. For better illustration, the semantic integration part is named DSI. In DSI, we design the DSPF feature fusion module and introduce the lightweight upsampling operator DySample to accomplish the feature alignment task.

Deep spatial pyramid fusion

In object detection tasks, especially small object detection, the problem of low feature map resolution and severe information loss is common. Traditional convolution layers process input images through sliding window operations, forming local receptive fields. However, the receptive field sizes of different convolution layers vary, and directly fusing these features introduces mismatched local information. BiFPN and ASFF improve detection performance by performing weighted summation across different feature maps. However, a similar approach struggles to solve the issue of mismatched receptive field sizes. Relatively SPPF has the potential to address the problem of varying receptive field sizes by segmenting and pooling feature maps at different scales, capturing information within varying local receptive fields. Inspired by the SPPF operation, we have fine-tuned the SPPF structure and designed the DSPF module. The default usage of DSPF differs from that of SPPF: In most object detection models, SPPF is added at the last layer of the backbone network to aggregate the global information of deep features. Since deep-layer feature maps contain rich semantic information, they are more suitable for detecting larger objects. However, in our approach, we place the DSPF module in the semantic integration part for feature fusion. This usage is more suitable for small object detection compared to conventional object detection models.

For a given input feature map $F \in R^{C \times H \times W}$ (where C is the number of channels, and H and W are the height and width of the feature map, respectively), SPPF extracts multi-scale information through pooling operations at

different scales $F_{Pool}^{(i)} = maxPool(F^{i-1}, p)$. Specifically, it applies maximum pooling to the input feature map *maxPool*(*F*, *p*),where p is the size of the pooling kernel, typically set to 5 by default. Finally, the outputs from each pooling layer are concatenated and fused:

$$F_{out} = Conv_{1\times1}([F^1, F^2, \ldots, F^i]) \tag{1}$$

Where F_{out} represents the final output feature map. $Conv_{1\times1}$indicates pointwise convolution and[,] represents channel concatenation. However, when dealing with small objects, default pooling operations may suppress the detailed information of small objects, leading to a decrease in detection performance. On the other hand, convolution operations, under the action of the convolution kernel, can maintain the transmission of local features. Additionally, we observe that the *maxPool* operation does not involve interactions across channels, which is similar to depthwise convolution. The final step of SPPF uses pointwise convolution to fuse the concatenated features, which corresponds to the pointwise convolution used in depthwise separable convolution[35]. Inspired by this, we replace the maximum pooling operation in SPPF with depthwise convolution. Max pooling retains only the maximum value within a local region, which may result in the loss of important feature information. In contrast, depthwise separable convolution can more comprehensively extract and preserve local details, making it more advantageous for small object detection. Let the feature map processed by depthwise convolution be denoted as $F_{Dw}^{(i)}$, and its computation formula is as follows:

$$F_{Dw}^{(i)} = DepthwiseConv(F^{i-1}, w) \tag{2}$$

where *DepthwiseConv* represents the feature map after depthwise convolution, and *w* is the kernel size of the depthwise convolution. To reduce computational overhead, we set *w*=3. Afterward, the outputs of each depthwise convolution layer are concatenated and fused, which is consistent with (1).

To enhance the model's ability to capture information at different scales, we introduce dilated convolution, which increases the receptive field of the convolution kernel without adding extra computational cost. The dilation rate of dilated convolution is defined as $r_1, r_2, \ldots, r_k$. After introducing the dilation rate, certain modifications need to be made to (2):

$$F_{Dw\_d}^{(i)} = DepthwiseConv_d(F_d^{i-1}, w, r_j) \tag{3}$$

Where $F_{Dw\_d}^{(i)}$ represents the output feature map after processing with depthwise dilated convolution, $DepthwiseConv_d$ represents the depthwise dilated convolution operation, and r_j is the dilation rate. If we simply stack multiple dilated convolutions with the same dilation rate, we may lose the continuity of the captured information, which is highly detrimental to small object detection. As the dilation rate increases, the receptive field of the convolution kernel expands accordingly, allowing for better capture of long-range contextual information. However, an excessively large dilation rate may miss important local details, resulting in the loss of small object information. Conversely, if the dilation rate is too small, the receptive field becomes limited, making it difficult to capture the relationship between small objects and complex backgrounds. Moreover, simply stacking multiple dilated convolutions with the same dilation rate can lead to a loss of continuity in the captured information, which is particularly detrimental to small object detection. In small object detection tasks, both local features and global contextual information are crucial. Gradually increasing the dilation rate layer by layer helps the network learn more extensive contextual information from fine-grained local features. Through experiments (Refer to the ablation study section "Experiment on different dilation rates of dilated convolution".), we have found that progressively increasing the dilation rate of dilated convolutions layer by layer can better capture features at different scales, thereby significantly improving the detection accuracy of small objects.

As shown in Fig. 3, we apply three depthwise separable dilated convolutions with a stride of 1 and a kernel size of 3, stacked with dilation rates r_1, r_2, r_3 of 1,2, and 3, respectively. The number of convolution groups is equal to the input channel count. These depthwise separable dilated convolutions process the feature map, and finally, the original feature map is concatenated with the feature map processed by dilated convolutions. The DSPF module processes features at different scales through spatial pyramid pooling and dilated convolutions, capturing multi-level information of small objects and avoiding information loss in multi-scale feature fusion as seen in traditional methods. The depthwise separable convolution significantly reduces computational cost, avoiding the high computational complexity of traditional convolutions. Dilated convolution expands the receptive field and enhances the fusion of multi-scale information, improving the network's sensitivity and detection ability for small objects.

DySample

Upsampling Channels are used to restore low-resolution feature maps, obtained through convolution, to high resolution. This helps to preserve spatial detail information crucial for small object detection. In L-FPN, the upsampling channels are employed to transfer deep semantic information to the shallow layers.

To improve model inference speed, this paper uses a lightweight upsampler, DySample (Fig. 4), for upsampling operations. DySample bypasses kernel-based examples and returns to the essence of upsampling by employing point sampling, dynamically adjusting the sampling point positions based on the content of the input image. DySample uses a sampling point generator to create dynamic sampling offsets for each pixel, and then adds the offsets to the original grid positions to obtain the sampling set. The input feature X is resampled through the grid sampling function to produce the upsampled feature X'. The sampling point generator controls whether to

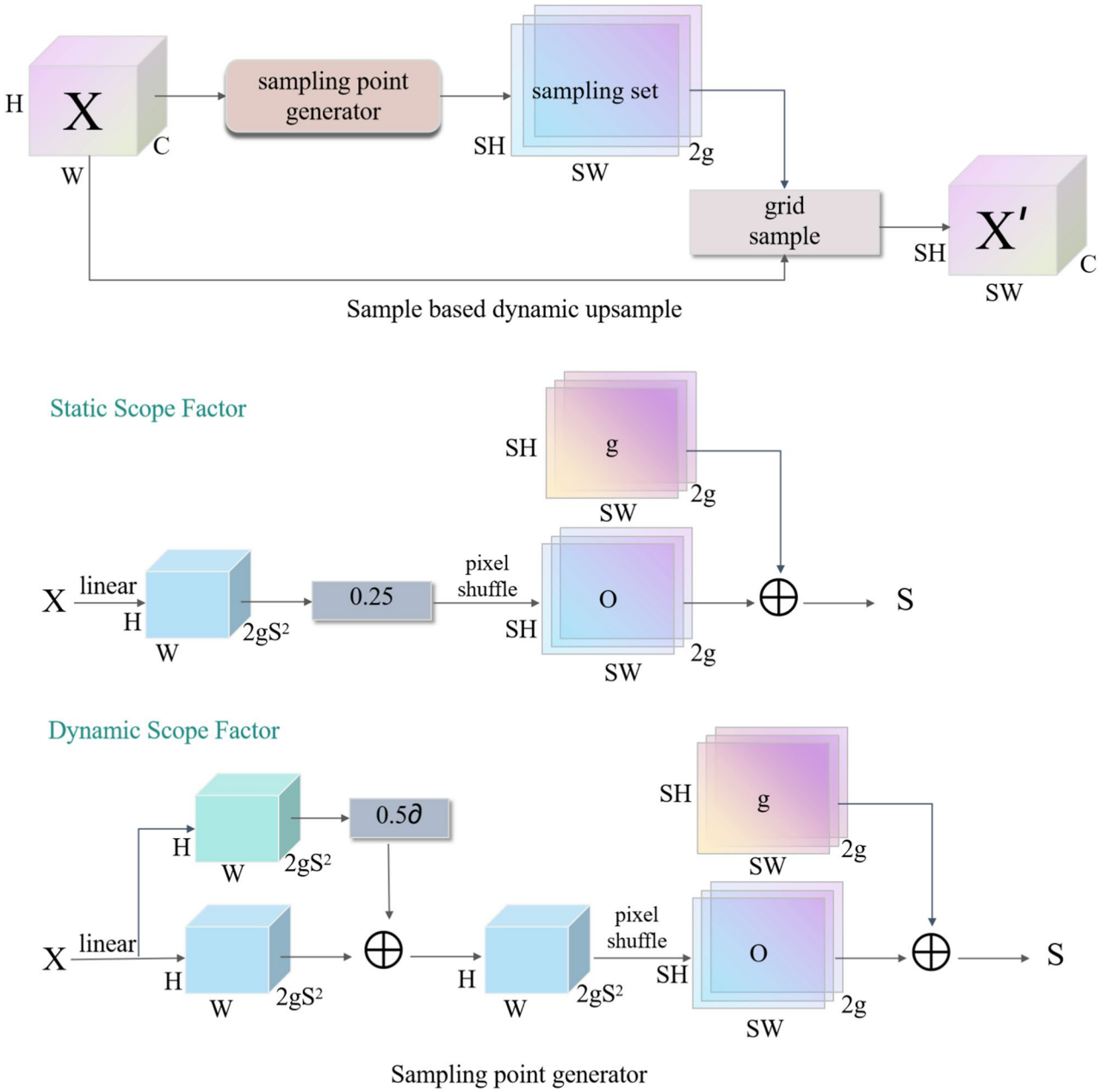


Fig. 4. DySample Structure. The input feature, upsampled feature, generated offset, and original grid are represented by X, X′, o, and g, respectively. "∂" represents the Sigmoid function, and is the upsampling scale factor.

adjust the offset using a range factor in two ways: "static range factor" and "dynamic range factor." In the version with a static range factor, for a feature map of size $C \times H \times W$, the offset is generated via a linear layer and directly applied for pixel transformation to produce a $2 \times S \cdot H \times S \cdot W$ feature map, where "2" represents the x and y coordinates, and s is the upsampling scale factor. In the dynamic range factor version, a content-aware method generates the range factor first, followed by the generation of the offset through a linear layer and pixel transformation.

Small objects often occupy only a few pixels on feature maps, making it difficult to distinguish target regions from the background during small object detection. Fixed interpolation upsampling methods (such as bilinear interpolation) apply the same formula uniformly to all pixels and cannot perceive the feature differences among pixels in the feature map. Compared to fixed interpolation, DySample adaptively adjusts the upsampling weights based on the input features, allowing the model to focus more on target regions and reduce background interference. This helps improve the model's ability to perceive small objects. Additionally, DySample is a lightweight upsampling method that enhances upsampling quality and small object detection performance without significantly increasing computational cost.

In the L-FPN architecture, we further enhance the expressive power of the feature maps by incorporating the advantages of DySample. L-FPN itself, through multi-scale feature fusion and contextual information enhancement, is already capable of capturing targets at different scales effectively. DySample plays a crucial role in this process. By dynamically adjusting the reference points for upsampling, DySample provides a more precise upsampling process for feature maps at different scales, thereby improving the network's ability to detect small objects. This is especially effective during the fusion of high-resolution feature maps, where it avoids the information loss that traditional methods may experience during information transfer.

Dual-phase asymptotic feature fusion mechanism

In L-FPN, we adopt a dual-head progressive fusion approach to sequentially fuse the feature maps output by each layer of the backbone network, forming a feature network primarily driven by shallow detail information flow. Progressive fusion is a layer-by-layer refinement feature fusion method that avoids the complexity and information loss introduced by directly integrating large-scale features. The typical feature pyramid network using progressive fusion is AFPN (Fig. 5), which aims to provide detailed shallow feature information to the deep layers. However, it does not improve the network's ability to detect small objects. This insight inspired us to adjust the information flow so that deep semantics primarily inform the shallow features, similar to the classical FPN or U-Net information flow direction. As shown in Fig. 5, compared to AFPN, our design includes three key improvements:

Remove Multiple Deep Layer Fusion Stages: In AFPN, deep features undergo multiple fusion stages, which can cause deep features to have too much influence on the network's final decision. In our design, we retain only a single fusion operation between the outputs of layers P4 and P5, eliminating unnecessary intermediate fusion stages and preventing excessive deep feature fusion.

Parallel Fusion of Shallow and Deep Features: AFPN tends to gradually fuse deeper features in the later stages of the network. This design leads to a network with many deep stages in the latter part. However[4], points out that deep features are not suitable for detecting small objects in the final detection head. Our strategy is to parallelly fuse deep and shallow features in the early stages of the network, which generates "intermediate layer" features with a smaller semantic gap, thus preparing for feature fusion in later stages.

Sparsify Deep Layer Connections, Dense Shallow Layer Connections: We adjusted the connection strategy for deep layers by changing the original progressive dense connections to include dense connections between shallow layers. Specifically, AFPN first fuses two shallow features and gradually integrates deep features into the fusion process, achieving feature fusion between non-adjacent layers, which avoids feature information loss. However, there is a significant semantic gap between non-adjacent layers, whereas the semantic gap between adjacent layers is smaller. Therefore, direct feature interaction between non-adjacent layers with significantly different scales may cause fine-grained details to be covered by semantic information, leading to the information flow being dominated by deep layers, which is unfavorable for small object detection. In contrast, our design of sparse deep-layer and dense shallow-layer connections shifts the focus of the network to shallow features, which provide more detail information, making the network better suited for small object detection tasks.

Based on these three improvement ideas, we propose a dual-head progressive fusion approach. In the initial stage of fusion, we first merge deeper and shallower layers to neutralize the semantic gap. Specifically, let L_j^i represent the feature map at layer i^{th}, and j^{jh} stage below that layer (starting from BackBone). We first perform parallel fusion of the outputs from the Backbone:

$$L_1^3 = DSPF(f_{down}(L_0^2), L_0^3) \tag{4}$$

$$L_1^4 = DSPF(L_0^4, f_{up}(L_0^5)) \tag{5}$$

where $\{L_0^2, L_0^3, L_0^4, L_0^5\}$ correspond to the outputs of the Backbone $\{C_2, C_3, C_4, C_5\}$, represents the downsampling operation using stride convolution, and represents the upsampling operation (such as DySample). L_1^3 and L_1^4 are at the intermediate stage of the downsampling phase, where they help bridge the semantic gap.

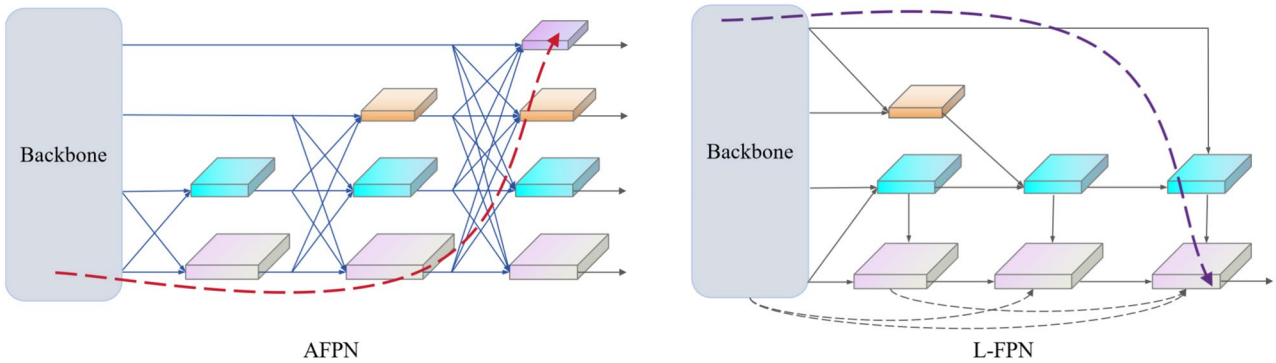


Fig. 5. AFPN and L-FPN. The blue arrows represent adaptive spatial fusion. The red dashed arrows and purple dashed arrows represent the information flow. The black dashed arrows represent skip connections.

In the later stage of fusion, we gradually merge the semantic information from the generated intermediate layers:

$$L_2^3 = DSPF(f_{up}(L_1^4), L_1^3) \tag{6}$$

$$L_3^3 = DSPF(f_{up}(L_0^5), L_2^3) \tag{7}$$

where the fusion L_2^3 occurs at the intermediate layers (L_1^4, L_1^3), while the further fusion combines the outputs (L_2^3, L_0^5) from the deepest layer of the Backbone and the previous stage's output from layer P3.

In DEI, the shallow layers progressively acquire the semantics from the deep layers. For L_i^2, when i=1, there are:

$$L_1^2 = f_{up}(L_1^3) \tag{8}$$

when i>1, there are:

$$L_i^2 = [f_{up}(L_i^3), L_1^2, L_2^2, \ldots, L_{i-1}^2] \tag{9}$$

where [,] represents stitching by channel dimension, represents adjusting the resolution of the feature map through upsampling. The initial fusion L_1^2 only contains the semantics of P2 and P3 layers, the second fusion contains the semantics of P2, P3, and P4 layers, and the final fusion L_3^2 contains the semantics of all layers. Through this progressively refined fusion process, we can effectively avoid semantic conflicts between shallow and deep layers and integrate multi-scale detailed information into a more powerful feature representation. By adjusting the feature fusion strategy, we improve the traditional fusion design of AFPN by adopting a dual-head progressive fusion, which better combines shallow details with deep semantic information, thereby enhancing the accuracy of small object detection. The sparse deep-layer connections and dense shallow-layer connections prevent deep-layer information from dominating the network's decision-making process, ensuring the preservation of detailed information and effectively improving the network's sensitivity and detection capability for small objects.

Decoupled feature extraction-semantic integration mechanism

In object detection tasks, shallow features contain rich detail information, while deep features encapsulate abundant semantic information. For most object detection networks, detail information and semantic information are inseparable, especially in small object detection scenarios. Traditional residual blocks play two roles in this process: on one hand, they extract features through multiple convolutional operations, and on the other hand, through hierarchical and residual connections, they form multi-scale feature representations to help integrate semantic information across different layers, addressing the semantic gap when fusing deep and shallow features. However, despite the effective integration of semantic information in deep networks through residual blocks, the low resolution of deep features and the lack of spatial details limit their contribution to small object detection. Particularly in small object detection, deep features often fail to provide sufficient spatial details and may negatively affect the network's performance. Therefore, we propose DEI, which weakens the feature extraction role of deep layers and enhances their semantic integration function, thereby more effectively combining shallow and deep features to improve the detection ability for small objects. In this mechanism, deep residual blocks are replaced by the DSPF module, which effectively integrates semantic information from different scales through multi-scale feature fusion. Unlike traditional residual blocks, which rely on extensive channel interactions for feature extraction, the DSPF module primarily uses multi-scale spatial pooling to fuse semantic features from different layers. This both avoids the semantic gap problem and efficiently transmits the multi-scale features extracted by the deep layers of the backbone network to the shallow layers.

Specifically, residual blocks play two main roles in traditional networks: (1) Feature extraction—extracting features through multi-layer convolution operations and channel interactions; (2) Semantic integration—forming multi-scale feature representations through a hierarchical design to effectively mitigate the semantic gap. We believe that in small object detection tasks, the primary responsibility for feature extraction has already been taken on by the backbone network and shallow features. In the FPN structure, deep features play a more significant role in integrating semantics. Therefore, in the deep layers of the FPN, we choose to use the DSPF module for semantic integration instead of relying on the computationally expensive residual blocks for feature extraction. We name this semantic integration part as DSI (Deep Semantic Integration). Let $\{C_2, C_3, C_4, C_5\}$ represent the feature sets at different scales output by each layer of the backbone network. In DSI, we use four DSPF modules to integrate semantic features from different scales and align the fused feature map to the same resolution as the C_2 layer using DySample. Finally, the integrated feature map is fed into the feature extraction layer.In the feature extraction layer, we use residual blocks to fuse the features with shallow features and extract rich detail features.

The DSPF module in DSI, through multi-scale spatial pooling operations, not only effectively integrates features from different scales but also preserves more spatial detail information, which is crucial for small object detection. Compared to traditional residual blocks, the DSPF module reduces channel interactions, lowers computational overhead, and enhances detection accuracy by strengthening semantic integration. Especially when dealing with small objects, DSPF effectively avoids the negative impact of deep features on small object detection, thus better preserving detail information and improving the model's overall detection capability. Experimental results show that this design, while ensuring detection accuracy, improves computational efficiency to some extent. Particularly in small object detection tasks, the DSPF module can effectively reduce the number

Datasets	Tiny	Small	Medium	Large
General Datasets				
MS COCO[36]	3.1%	9.7%	19.1%	68.1%
Pascal VOC[37]	0.1%	0.9%	5.8%	93.2%
Specialized Datasets				
TinyPerson[38]	80.7%	15.0%	3.5%	0.8%
VisDrone2019-DET[39]	32.7%	35.6%	22.6%	9.1%

Table 1. Distribution of large, medium, small, and very small objects in different datasets. "Tiny" refers to objects smaller than 16 × 16 in size."Small" refers to objects between 16 × 16 and 32 × 32 in size."Medium" refers to objects between 32 × 32 and 64 × 64 in size."Large" refers to objects larger than 64 × 64 in size.

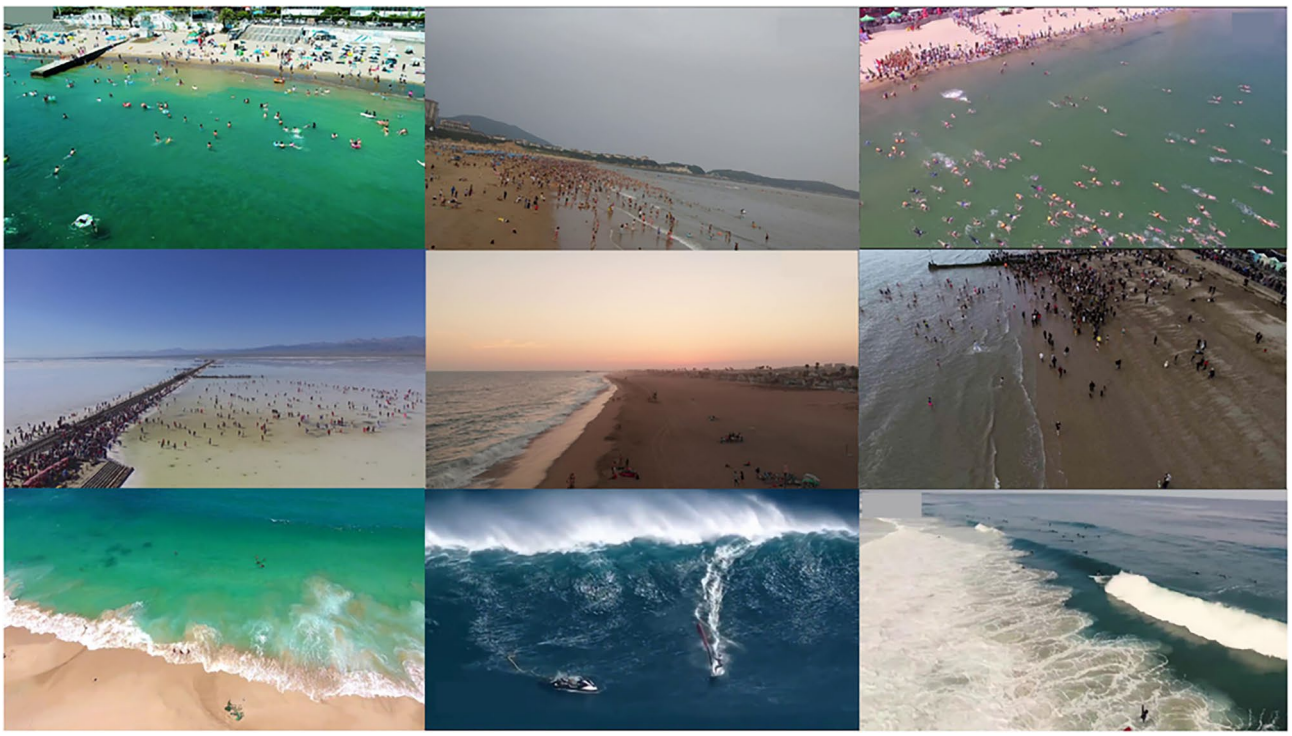


Fig. 6. TinyPerson dataset.

of parameters and enhance model efficiency compared to traditional residual block networks. Through this decoupling mechanism, we not only avoid the negative impact of deep features on small object detection but also effectively integrate features from different scales, enhancing small object detection capability while optimizing computational efficiency.

Experimental results

Datasets

As shown in Table 1, in conventional datasets, large objects dominate, while very small objects make up only a small proportion. However, in drone-captured images, very small and small objects dominate, with large objects occupying a negligible proportion. Therefore, we choose drone-captured images as the experimental dataset for this study. We selected the public datasets VisDrone2019 and TinyPerson to validate the effectiveness of our proposed method.

VisDrone2019

The VisDrone2019 dataset is large in scale, consisting of 10,209 high-resolution(2000 × 1500) images. Of these, 6,471 are used for training, 548 for validation, and 3,190 for testing. The dataset includes ten target categories: pedestrians, people, cars, vans, buses, trucks, motorcycles, bicycles, covered tricycles, and tricycles. Captured by a drone, the images have high resolution. Additionally, the targets are densely distributed and relatively small in size, making the dataset ideal for small target detection research.

TinyPerson

The TinyPerson dataset consists of 1,610 images, with 794 in the training set and 816 in the testing set. The dataset contains a total of 547,800 human targets, with target sizes ranging from 2 to 20 pixels. As shown in Fig. 6, the TinyPerson dataset includes two types of targets: "ocean people" (people in the sea) and "earthlings" (people on land). Since our model focuses on object detection, we treat both types of people as a single class: "people." This dataset is highly suitable for evaluating the performance of small object detection models, particularly in complex and dense scenes, where we use it to validate the model's detection capabilities.

Implementation details

All experiments were conducted on a Windows operating system. The experimental code was written in Python (3.9) and implemented using the PyTorch deep learning framework (version 2.4.0+cu121). The experiments were run on an NVIDIA 4060 Ti GPU workstation.

To evaluate the effectiveness of L-FPN, we chose YOLOv8n as our baseline model, with YOLOv8n+p2 (unless otherwise specified, the P2 layer in this section refers to the P2 layer in this section refers to the small object detection layer) as the reference. Compared to other models in the YOLO series, YOLOv8 features an updated architecture with greater scalability. YOLOv8n has lower computational complexity and fewer parameters, making it suitable for edge devices or scenarios with high real-time requirements. Therefore, we select YOLOv8n as our baseline model. Furthermore, since high-resolution shallow feature maps are crucial for small object detection, we incorporate the P2 layer into YOLOv8n and use YOLOv8n+P2 as our baseline model.

For training on the VisDrone2019 dataset,we adopt the SGD optimizer with a cosine annealing learning rate schedule. The initial learning rate is set to 0.01 and gradually decays to 0.0001 by applying a decay factor of 0.01. The training runs for 300 epochs with a batch size of 8 for both forward and backward propagation. During training, we employ data augmentation techniques such as Mosaic, Mixup, and flipping, and disable data augmentation during the last 10 epochs. When training on the TinyPerson dataset, to improve model accuracy and prevent memory overflow, we set the image size (imgsz) to 1024 and the batch size to 2. All other parameters remained consistent with the above settings. The evaluation metrics used in this study include precision (P), recall (R), average precision at an IoU threshold of 0.50 (mAP 50), and average precision calculated between IoU thresholds of 0.50 and 0.95 (with a step size of 0.05) (mAP 50-95).

This paper uses Precision (P), Recall (R), Mean Average Precision at IoU threshold 0.50 (mAP50), the mean average precision (mAP50-95),and F1 score(F1) calculated across IoU thresholds from 0.50 to 0.95 (with a step size of 0.05) as evaluation metrics.

Precision measures the accuracy of the model's detection, i.e., the proportion of true positive predictions out of all predicted positives.

$$P = \frac{TP}{TP + FP} \tag{10}$$

where TP denotes the number of true positive samples correctly predicted by the model, and FP represents the number of false positive samples incorrectly predicted as positive.

Recall is used to measure how many of the actual positive samples are correctly predicted as positive by the model.

$$R = \frac{TP}{TP + FN} \tag{11}$$

where FN denotes the number of samples that are actually positive but are incorrectly predicted as negative by the model.

MAP50 refers to the average detection precision of the model across different categories, evaluated at an IoU threshold of 0.50.

$$mAP50 = \frac{AP_1 + AP_2 + \cdots + AP_n}{n} \tag{12}$$

where AP_i is the maximum value of precision (P) within each interval.

MAP50-95 considers the model's detection performance at different IoU thresholds and provides a comprehensive evaluation across all categories.

$$mAP50 - 95_C = \frac{\sum_{i=0.5}^{0.95} AP_i}{n} \tag{13}$$

$$mAP50 - 95 = \frac{\sum_{j=1}^{n} mAP50_C}{n} \tag{14}$$

where $mAP50_C$ is the average precision at the $mAP50 - 95$ threshold, and i represents the different IoU thresholds.

F1 score, articulated in Eq. (15), serves as the harmonic average of Precision and Recall, providing a comprehensive evaluation of algorithmic performance.

$$F1 = \frac{2PR}{P+R} \tag{15}$$

Additionally, we use Floating Point Operations (FLOPs) and the number of parameters (Params) to evaluate the model's complexity and size. FLOPs represent the number of floating-point operations required per second, where a smaller FLOPs value indicates lower computational complexity. Similarly, a smaller number of parameters means the model requires less storage space.

Experiments on the VisDrone2019 dataset

Comparisons with previous methods

To validate the effectiveness of BPD-YOLO, we compared it with existing advanced models on the VisDorne2019 dataset. The experimental results are shown in Table 2. Specifically, by comparing the experimental results of DMNet, YOLC, YOLO v3-SPP3, GFL, RetinaNet, Faster R-CNN, and BPD-YOLOn, BPD-YOLOs, it can be observed that BPD-YOLOn outperforms RetinaNet and Faster R-CNN in detection accuracy, while its computational cost is much lower than both. Specifically, compared to RetinaNet, BPD-YOLO achieves improvements of 9% and 7.7% on mAP50 and mAP50-95 respectively, while reducing parameters and computational cost by 41.24 million and 557.17 GFLOPs. Compared to Faster R-CNN, BPD-YOLO improves mAP50 and mAP50-95 by 2.3% and 3.1% respectively, with reductions of 28.47 million parameters and 292.65 GFLOPs in computation.

Compared to lightweight object detection algorithms such as C3TB-YOLOv5, GCGE-YOLO, and YOLOv7-Tiny, BPD-YOLO also demonstrates outstanding performance. Specifically, compared to YOLOv7-Tiny, BPD-YOLO achieves higher detection accuracy while having lower computational cost and fewer parameters. Although C3TB-YOLOv5 exhibits slightly higher detection accuracy than BPD-YOLO, the latter significantly reduces both parameters and computation, with a decrease of 6.5 million parameters and 8.3 GFLOPs. Compared to GCGE-YOLO, BPD-YOLO reduces parameters by 3 million and improves detection accuracy by 4% and 3.4% on mAP50 and mAP50-95, respectively.Compared with FFCA-YOLOn, BPD-YOLOn achieves higher detection accuracy and lower computational cost.

Additionally, compared to classical improved FPNs such as AFPN and BiFPN, our method also demonstrates significant performance advantages. Compared with YOLOv8n+P2+AFPN, BPD-YOLO reduces parameters by 69.8% and computational cost by 39.7%, with only slight decreases in detection accuracy: mAP50 drops by 0.2% and mAP50-95 by 0.5%. Compared to YOLOv8n+P2+BiFPN, BPD-YOLO achieves improvements of 2.8% and 1.5% on mAP50 and mAP50-95 respectively, while reducing parameters and computation by 1.44 million and 0.9 GFLOPs.Compared with TPH-YOLO, although BPD-YOLOn increases the computational load by 5.5G, its detection accuracy improves by 7.7% on mAP50 and 6.6% on mAP50-95.

Methods	Imgsz	P	R	F1	mAP50	mAP50-95	Params	GFLOPs
DMNet[40]	1333*800	-	-	-	0.493	0.294	-	-
YOLC[41]	1333*800	-	-	-	0.524	0.297	-	-
YOLOv3-SPP3[42]	1333*800	-	-	-	-	0.264	63.9M	284.10
GFL[43]	1333*800	-	-	-	0.5	0.284	42.52M	524.95
RetinaNet	1333*800	-	-	-	0.369	0.202	42.74M	586.77
Faster R-CNN[44]	1333*800	-	-	-	0.436	0.248	29.97M	322.25
C3TB-YOLOv5[45]	640*640	-	-	-	0.383	0.22	8.0M	19.7
GCGE-YOLO[46]	640*640	-	-	-	0.341	0.192	4.5M	10.8
YOLOv7-Tiny	640*640	0.471	0.366	-	0.35	-	6.03M	13.3
FFCA-YOLOn	640*640	0.474	0.345	0.39	0.349	0.193	7.04M	15.8
TPH-YOLOn	640*640	0.397	0.315	0.33	0.304	0.16	2.11M	5.9
YOLOv8s	640*640	0.51	0.399	0.44	0.399	0.241	11.12M	28.5
YOLOv8n	640*640	0.432	0.333	-	0.326	0.188	3.00M	8.1
YOLOv8n+P2	640*640	0.467	0.351	0.39	0.353	0.212	2.92M	12.2
YOLOv8s+P2	640*640	0.522	0.433	0.47	0.429	0.262	10.62M	36.7
YOLOv8n+P2+AFPN	640*640	0.497	0.376	0.42	0.383	0.231	4.96M	18.9
YOLOv8n+P2+BiFPN	640*640	0.461	0.361	0.37	0.353	0.211	2.94M	12.3
BPD-YOLOn(Ours)	640*640	0.494	0.382	0.42	0.381	0.226	1.50M	11.4
BPD-YOLOs(Ours)	640*640	0.542	0.451	0.49	0.45	0.274	5.76M	36.7
BPD-YOLOn(Ours)	1333*800	0.538	0.458	0.49	0.459	0.279	1.50M	29.6
BPD-YOLOs(Ours)	1333*800	0.6	0.541	0.56	0.542	0.338	5.76M	95.4

Table 2. Performance comparison of different models on the visdrone2019 dataset."-" indicates that the result was not reported or is unavailable. For the models using BPD-YOLO and the Imgsz of 1333×800 in the experiments, to prevent out-of-memory errors, the batch size was set to 1.

The above experimental results indicate that BPD-YOLO achieves a good balance between accuracy and efficiency, making it especially suitable for applications that demand high precision but have limited computational resources.

Comparisons with baseline

Compared to the baseline model YOLOv8n+p2 (Table 2), BPD-YOLOn improves by 2.8% in mAP50, 1.4% in mAP50-95, and reduces computational cost by 0.8 GFLOPs. Compared to the original model YOLOv8n, BPD-YOLOn also improves by 5.5% in mAP50 and 3.8% in mAP50-95. Figure 7 shows the detection performance comparison between FPN and BPD-YOLO on the VisDrone2019 test set. In image group (a), compared to FPN, BPD-YOLO significantly reduces missed detections, indicating that BPD-YOLO is relatively sensitive to small objects in complex backgrounds. In image group (b), FPN misses large vehicles and incorrectly detects small objects in the distance, while BPD-YOLO's detection results align with the ground truth. This suggests that BPD-YOLO can effectively integrate multi-scale features and balance the detection of both large and small objects. In image group (c), BPD-YOLO accurately identifies occluded objects, demonstrating its superiority in handling complex feature interactions. Moreover, the heatmap indicates that BPD-YOLO exhibits strong sensitivity to small objects.

As shown in Figure 8, the normalized confusion matrices illustrate the performance comparison between the baseline and our proposed BPD-YOLO. By examining the diagonal elements of the normalized confusion

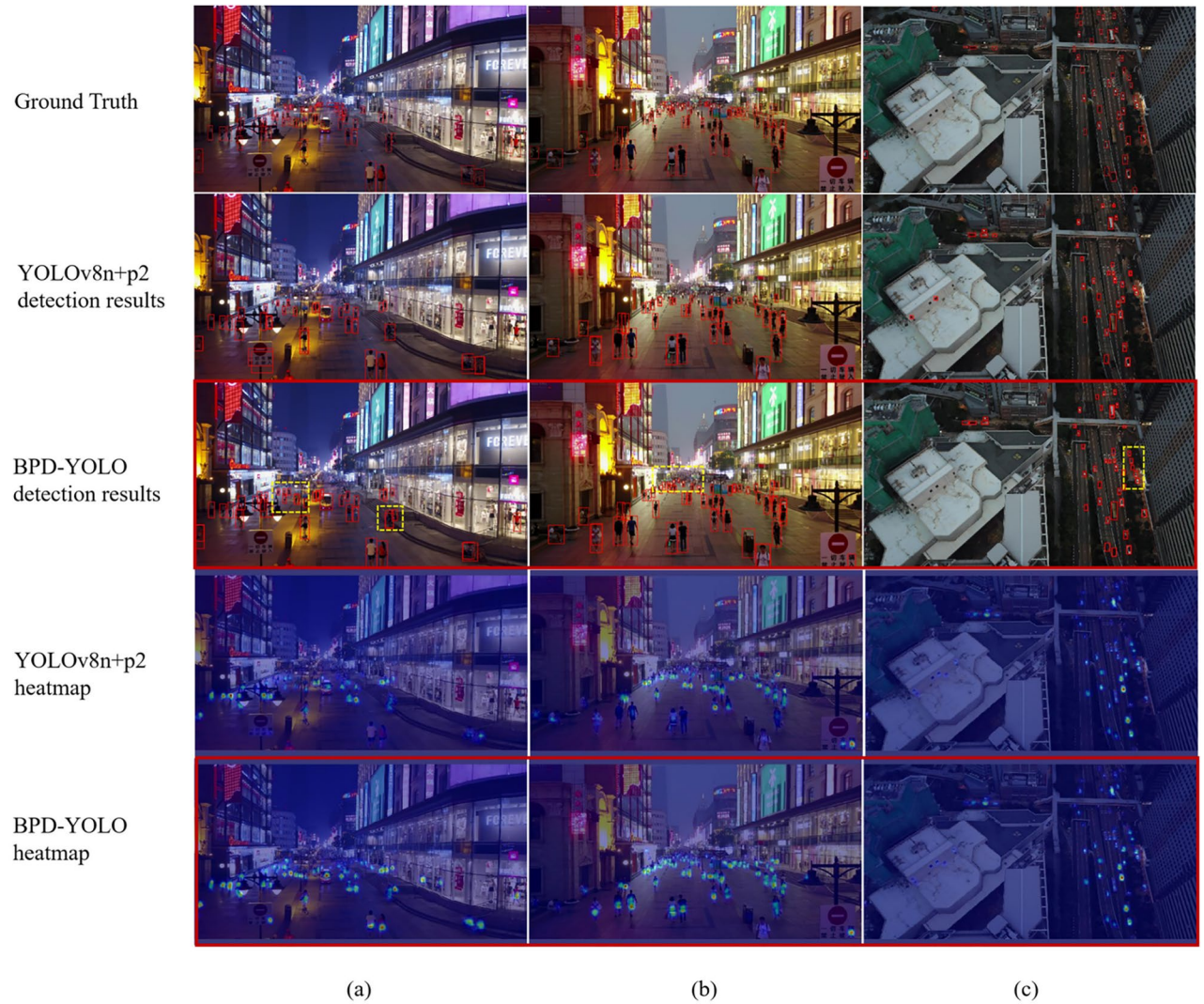
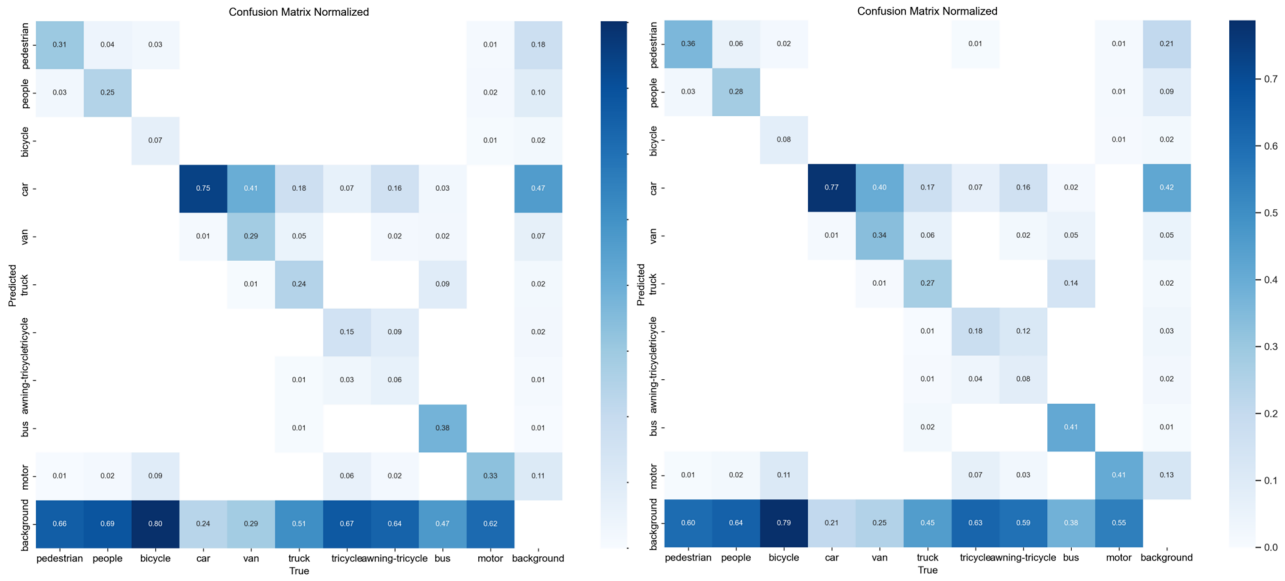


Fig. 7. Visualization of the Comparison Results of Different Methods on the VisDrone2019 Test Set. For each test image, we display the ground truth, FPN, and BPD-YOLO detection results. The yellow dashed boxes indicate the small object detection performance under different scenarios. The yellow dashed boxes highlight the performance of small object detection in different scenarios. In the images of groups (**a**) and (**b**), the dense crowds within the yellow boxes were missed by YOLOv8n+P2, whereas our BPD-YOLO successfully detected these crowded groups. In group (**c**), the vehicles inside the yellow boxes were not correctly detected by YOLOv8n+P2, but BPD-YOLO achieved more accurate detection.



YOLOv8n+p2(Baseline) BPD-YOLOn

Fig. 8. Normalized confusion matrices of the baseline model (left) and the proposed BPD-YOLO method (right). The results show that BPD-YOLO improves class-wise prediction accuracy and reduces confusion between similar categories.

Methods	P	R	F1	mAP50	mAP50-95	Params	GFLOPs
YOLOv5n	0.356	0.284	0.30	0.262	0.132	1.77M	4.2
YOLOv5s	0.438	0.344	0.38	0.333	0.182	7.04M	15.8
YOLOv5n+P2	0.391	0.308	0.33	0.298	0.157	1.81M	4.9
YOLOv5s+P2	0.486	0.374	0.42	0.379	0.211	7.18M	18.7
YOLOv5n+L-FPN	0.437	0.344	0.38	0.336	0.174	1.14M	6.2
YOLOv5s+L-FPN	0.529	0.422	0.46	0.432	0.241	4.67M	25.3
BPD-YOLOn(Ours)	0.494	0.382	0.42	0.381	0.226	1.50M	11.4
BPD-YOLOs(Ours)	0.542	0.451	0.49	0.45	0.274	5.76M	36.7

Table 3. Yolov5 series experimental results on the visdorne2019 dataset.

matrix, it can be observed that BPD-YOLO achieves higher detection accuracy across all categories compared to the baseline model. The most significant improvement is seen in the motor category, with an increase of 0.08, followed by pedestrian and van, each with an improvement of 0.05. These results indicate that BPD-YOLO possesses stronger discriminative capability and a lower false detection rate when handling small objects in complex scenes, further validating its effectiveness in multi-class object detection tasks.

Comparisons with YOLOv8s

It is worth noting that, compared to YOLOv8s (Table 2), BPD-YOLOn reduces the computational cost by 60% (from 28.5 GFLOPs to 11.4 GFLOPs), with minimal loss in mAP50 and mAP50-95, essentially maintaining the model's performance. This demonstrates that BPD-YOLO offers more efficient inference speed and significantly reduces computational cost without sacrificing detection accuracy. Furthermore, YOLOv8s+p2 and BPD-YOLOs have the same computational cost, but BPD-YOLOs achieves a 2.1% improvement in mAP50 and a 1.2% improvement in mAP50-95. This strongly indicates that our BPD-YOLO effectively extracts the detailed information of small objects in shallow layers and combines deep and shallow information efficiently, leading to a significant improvement in detection accuracy.

Comparisons with YOLOv5 series

As shown in Table 3, BPD-YOLOn and BPD-YOLOs achieve significantly higher detection accuracy compared to YOLOv5n+P2 and YOLOv5s+L-FPN. Moreover, compared to the baseline models of the YOLOv5 series (YOLOv5n+P2 and YOLOv5s+L-FPN), both YOLOv5n+L-FPN and YOLOv5s+L-FPN show notable improvements in accuracy. Specifically, YOLOv5n+L-FPN boosts mAP50 and mAP50-95 by 3.8% and 1.7%, respectively, while YOLOv5s+L-FPN achieves improvements of 5.3% and 3.0%, respectively. These results

Methods	P	R	F1	mAP50	mAP50-95	Params	GFLOPs
YOLOv10n+P2	0.447	0.353	0.39	0.347	0.212	2.9M	15.5
YOLOv10n+L-FPN	0.466	0.371	0.40	0.365	0.219	1.6M	13.7
BPD-YOLOn(Ours)	**0.494**	**0.382**	**0.42**	**0.381**	**0.226**	**1.50M**	**11.4**

Table 4. Yolov10 series experimental results on the visdorne2019 dataset.

Methods	P	R	F1	mAP50	mAP50-95	Params	GFLOPs
YOLOv8n+P2+AFPN	0.542	0.4	0.46	**0.392**	0.138	6.05M	79.6
YOLOv8n+P2+BiFPN	0.524	0.407	0.46	0.384	0.135	2.34M	49.7
YOLOv8n+P2	0.524	0.395	0.45	0.38	0.134	2.93M	31.7
BPD-YOLO(Ours)	**0.543**	**0.411**	0.47	0.391	**0.136**	**1.50M**	**30.9**

Table 5. Experimental results on the tinyperson dataset.

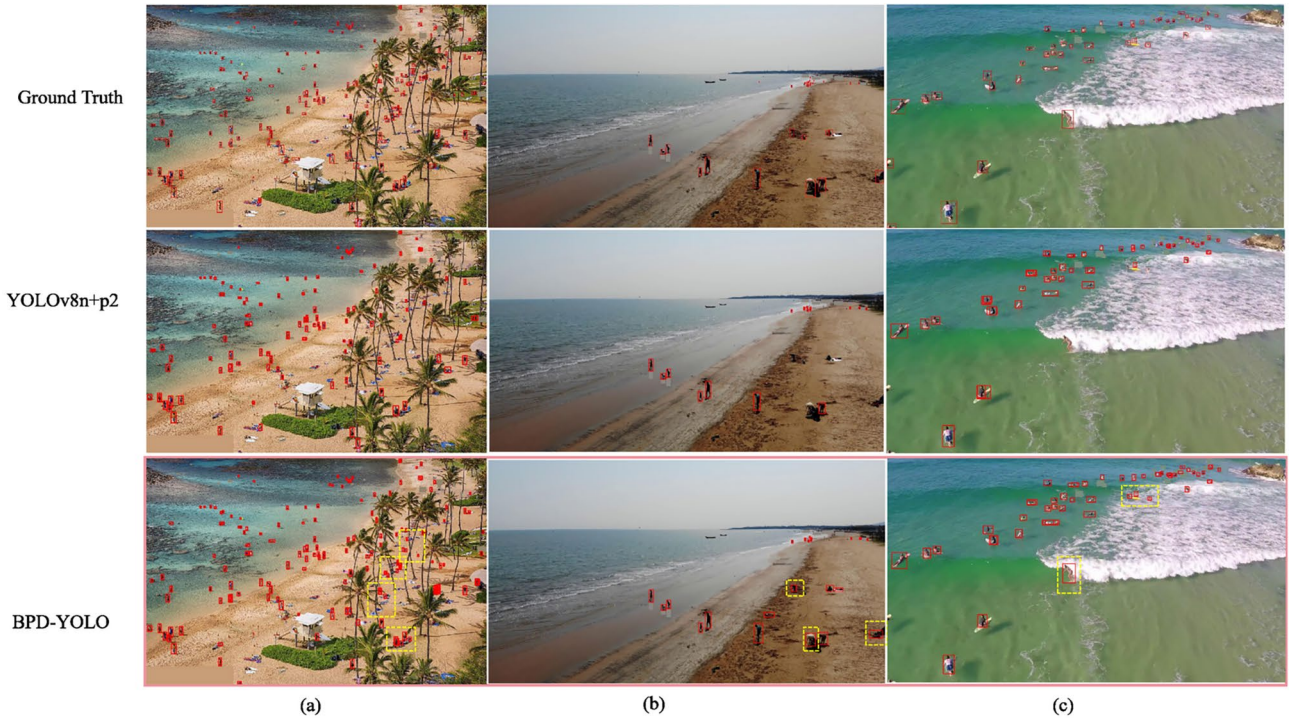


Fig. 9. Visualization of Comparison Results Between Different Methods on the TinyPerson Dataset Test Set. For each test image, we show the ground truth, YOLOv8n+p2, and BPD-YOLO detection results. The yellow dashed boxes highlight the performance of small object detection in dense crowds with complex backgrounds. The people within the yellow boxes in the image were not detected by YOLOv8n+P2, whereas our BPD-YOLO was able to successfully and accurately identify them.

demonstrate that L-FPN can effectively capture fine-grained details of small objects in small object detection scenarios.

Comparisons with YOLOv10 series

The experimental results are shown in Table 4. Compared to YOLOv10n+P2, BPD-YOLOn improves the mAP50 and mAP50-95 metrics by 3.4% and 1.4%, respectively, while reducing the number of parameters and computational cost by 1.4M and 4.1 GFLOPs. Similarly, compared to YOLOv10n+P2, YOLOv10n+L-FPN increases mAP50 and mAP50-95 by 1.8% and 0.7%, while reducing Params and GFLOPs by 1.3M and 1.8, respectively. These results demonstrate that L-FPN enhances detection accuracy while simultaneously reducing both parameter count and computational complexity.

In summary, BPD-YOLO delivers outstanding performance in small object detection, achieving a well-balanced trade-off between detection accuracy and computational efficiency.

Backbone	Methods	P	R	mAP50	mAP50-95	Params	GFLOPs
DarkNet[48]	FPN+p2	0.467	0.351	0.353	0.212	2.92M	12.2
	L-FPN	**0.49**	**0.384**	**0.381**	**0.226**	**1.50M**	**11.4**
FasterNet[49]	FPN+p2	0.478	0.368	0.369	0.218	4.10M	**15.0**
	L-FPN	**0.494**	**0.383**	**0.384**	**0.226**	**2.76M**	15.5
MobileNetV4[50]	FPN+p2	0.446	0.361	0.352	0.21	5.63M	26.8
	L-FPN	**0.469**	**0.364**	**0.362**	**0.215**	**2.96M**	**24.9**
EfficientFormerV2[51]	FPN+p2	0.494	0.385	0.388	**0.233**	5.03M	15.9
	L-FPN	**0.503**	**0.393**	**0.391**	0.232	**3.54M**	**13.8**

Table 6. Experimental results for different backbone + L-FPN combinations.

Base model	Methods	P	R	mAP50	mAP50-95	Params	GFLOPs
YOLOv8n	AFPN	**0.497**	0.376	**0.383**	**0.231**	4.96M	18.9
	BiFPN	0.461	0.361	0.353	0.211	2.94M	12.3
	U-Net	0.456	0.351	0.344	0.201	1.54M	**8.6**
	Unet++	0.475	0.37	0.37	0.22	1.6M	10.2
	Unet++(DySample)	0.484	0.381	0.376	0.223	1.7M	11.6
	FPN	0.467	0.351	0.353	0.212	2.92M	12.2
	L-FPN	0.494	**0.382**	0.381	0.226	**1.50M**	11.4

Table 7. Experimental results for same backbone + different network architectures.All comparison experiments, except L-FPN, use the p2 detection head.

Experiment on the TinyPerson dataset

The experimental results are shown in Table 5. On the TinyPerson dataset, compared to YOLOv8n+p2, BPD-YOLOn reduces the number of parameters by 1.43M and the computational cost by 0.8GFLOPs, while improving mAP50 and mAP50-95 by 1.1% and 0.2%, respectively. Figure 9 shows the performance comparison between YOLOv8n+p2 and BPD-YOLO on the TinyPerson test set. From the figure, it is clear that BPD-YOLO has higher accuracy when detecting densely distributed tiny targets. In the images of group (b), YOLOv8n+p2 exhibits significant missed detections and struggles with tiny target detection, while BPD-YOLO's results are much closer to the ground truth, with a noticeable reduction in missed detections. In groups (a) and (c), where the background is more complex, YOLOv8n+p2 suffers from substantial false detections and missed detections, with many overlapping detection boxes. In contrast, BPD-YOLO is able to effectively capture multi-level features that distinguish between the background and the targets, thereby improving detection accuracy.

Ablation study

To verify the effectiveness of the methods proposed in this paper, we conducted ablation experiments on the VisDrone2019 dataset. The image size (imgsz) was set to 640 for all experiments. Unless stated otherwise, the remaining parameters were set as described in Section Implementation details.

Ablation experiment on L-FPN

We performed an ablation study on L-FPN based on YOLOv8n. In this experiment, we replaced the FPN of YOLOv8n with L-FPN and compared their performance when combined with different backbones. The experimental results are shown in Table 6. As observed, L-FPN consistently outperforms FPN across different backbone configurations. Notably, for the EfficientFormerV2 backbone, which is based on Transformer[47] architecture, the computational resources are primarily allocated to medium and high-level semantic features. This limits the Transformer's ability to extract fine-grained, high-resolution details. As a result, the performance gap between FPN and L-FPN is minimal in this experiment, with mAP50 of 0.388 and 0.391, and mAP50-95 of 0.233 and 0.232 for FPN and L-FPN, respectively. However, overall, regardless of the backbone architecture, L-FPN achieves higher detection accuracy and lower computational cost, demonstrating the robustness of L-FPN across various network structures.

Experiments with different network architectures

We further validated the effectiveness of L-FPN by combining the same backbone network (using the YOLOv8 backbone) with different feature fusion network structures. It should be noted that, to focus on the comparison of network architecture and feature fusion methods, we did not strictly replicate the specific details of the original implementations of the compared methods, but instead uniformly combined them with the baseline model YOLO for the experiments. For example, in all implemented models, we uniformly replaced the feature extraction module with the C2f module to ensure fairness and consistency in the comparison. The experimental results are shown in Table 7 Compared to AFPN, L-YOLOv8n reduces computational cost by 7.5 GFLOPs, while only decreasing mAP50 and mAP50-95 by 0.2% and 0.5%, respectively. This indicates that our L-FPN,

Methods	P	R	mAP50	mAP50-95	Params	GFLOPs
FPN+SPPF	0.467	0.351	0.353	0.212	2.92M	12.2
FPN+DSPF	0.447	0.373	0.358	0.215	2.9M	12.2
L-FPN+Add	0.462	0.364	0.354	0.206	**1.37M**	**10.0**
L-FPN+C2f	0.47	0.379	0.376	0.224	1.53M	11.4
L-FPN+SPPF	0.457	0.377	0.365	0.214	1.48M	11.0
L-FPN+DSPF	**0.494**	**0.382**	**0.381**	**0.226**	1.50M	11.4

Table 8. DSPF ablation experiment results.

Structure	Upsampling	P	R	mAP@50	mAP@50:95	Params	GFLOPs
L-FPN	Bilinear	0.482	0.366	0.369	0.217	**1.46M**	**10.4**
	Nearest	0.472	0.368	0.368	0.218	**1.46M**	**10.4**
	CARAFE[52]	0.490	**0.384**	**0.381**	**0.226**	1.56M	11.6
	DySample	**0.494**	0.382	**0.381**	**0.226**	1.50M	11.4
FPN+P2	Nearest	**0.467**	0.351	0.353	0.212	**2.92M**	**12.2**
	DySample	0.460	**0.361**	**0.355**	**0.213**	3.03M	12.9

Table 9. Results of different upsampling methods experiments.

by optimizing the feature fusion approach, improves detection accuracy while saving computational resources, allowing L-FPN to perform well even in resource-constrained environments. Compared to the Unet++ method with improved upsampling using DySample, L-FPN achieves a higher mAP50 by 0.5%, while having fewer parameters and lower computational cost. This shows that L-FPN integrates more effectively with the improved upsampling method, DySample.

DSPF ablation experiment

We conducted various ablation experiments to evaluate the multiple functionalities of DSPF: First, we replaced the default SPPF module in the baseline model with DSPF to compare their multi-scale feature extraction capabilities. Additionally, while applying the feature extraction-semantic integration decoupling mechanism of L-FPN, we replaced various modules in the deeper layers of the network to compare their semantic integration performance. The experimental results are shown in Table 8.

In terms of multi-scale feature extraction, DSPF outperforms SPPF in detection performance. Replacing the original YOLOv8's SPPF with our DSPF leads to a 0.5% improvement in mAP50 and a 0.3% improvement in mAP50-95. In the evaluation of semantic fusion performance within the L-FPN framework, DSPF also demonstrates strong results: replacing SPPF with DSPF yields a 1.6% increase in mAP50, and replacing the residual-block-based C2f module with DSPF results in a 0.5% gain in mAP50.

Experimental results show that DSPF improves small object detection accuracy both when applied at the end of the backbone for multi-scale feature extraction and within the L-FPN for semantic fusion. Moreover, compared to Add and C2f, DSPF consistently achieves higher detection accuracy. This indicates that DSPF can effectively perform multi-scale feature fusion and feature extraction without introducing additional computational overhead, thereby enhancing the performance of small object detection.

Different upsampling methods experiment

In this section, we conducted experiments to investigate the impact of different upsampling methods on model performance, and based on the results, selected the most suitable upsampling method. The experimental results are shown in Table 9. Traditional interpolation upsampling methods (Bilinear Interpolation and Nearest Neighbor Interpolation) showed no significant difference in performance. Although they require less computational cost compared to the other two upsampling methods, their accuracy is notably lower. In contrast, DySample and CARAFE significantly outperform interpolation-based upsampling methods in terms of accuracy. Among them, DySample achieves almost the same accuracy as CARAFE while maintaining a lower computational cost. Moreover, when L-FPN adopts DySample as the upsampling method, it achieves a 1.3% improvement in mAP@50 and a 0.8% increase in mAP@50:95 compared to using Nearest Neighbor. Meanwhile, replacing the upsampling method in the original FPN with DySample leads to a 0.2% gain in mAP@50 and a 0.1% gain in mAP@50:95, with an additional computational cost of 0.7 GFLOPs. These results not only demonstrate that DySample offers better detection accuracy for small objects, but also indicate that DySample contributes more significantly to the performance of our proposed method (L-FPN) than to the original FPN, further validating the effectiveness and suitability of DySample within our framework.

From the perspectives of both detection accuracy and computational cost, DySample is the most suitable choice; therefore, we select DySample as the model's upsampling method.

Dilation rate	P	R	mAP50	mAP50-95
d=(1,2,4)	0.492	0.379	0.38	0.225
d=(2,2,2)	0.469	0.379	0.373	0.221
d=(3,2,1)	0.491	0.371	0.375	0.222
d=(1,2,3)	**0.494**	**0.382**	**0.381**	**0.226**

Table 10. Results of experiments with different dilation rates.

Parameter	P	R	mAP50	mAP50-95	Params	GFLOPs
g=1	0.487	**0.392**	**0.387**	**0.231**	1.93M	15.9
g=c	**0.494**	0.382	0.381	0.226	**1.5M**	**11.4**

Table 11. Results of experiments with depthwise separable convolution.When g=1 (where g is the number of groups), dilated convolution is applied; and when g=c (where c is the number of input channels), depthwise separable dilated convolution is used.

Methods	FPS
YOLOv8n+FPN+P2	125
YOLOv8s+FPN+P2	148
YOLOv8n+P2+AFPN	95
YOLOv8n+P2+BiFPN	90
YOLOv8n+P2+U-Net	**154**
YOLOv8n+P2+Unet++	116
BPD-YOLOn(Ours)	119
BPD-YOLOs(Ours)	115
FasterNet+FPN+P2	132
FasterNet+L-FPN(Ours)	108
MobileNetV4+FPN+P2	123
MobileNetV4+L-FPN(Ours)	105

Table 12. Frames Per Second(FPS) of each model.FPS refers to the number of frames a model can infer per second. Generally, depending on the specific application, real-time object detection models should be capable of processing video frames at a speed of at least 20-30 FPS to align with practical task requirements.

Experiment on different dilation rates of dilated convolution
In this section, we conducted experiments on different dilation rates of dilated convolution to evaluate the impact of varying dilation rates on the experimental results. As shown in Table 10, different dilation rate combinations did not affect the model's computational cost, but they had varying impacts on the model's accuracy. When the dilation rates were set to (1, 2, 3), the model achieved the highest accuracy, with mAP50 and mAP50-95 reaching 0.381 and 0.226, respectively, showing a significant improvement over other combinations. Therefore, we set the dilation rates of the dilated convolution in the feature fusion module DSPF to 1, 2, and 3.

Experiment on depthwise separable convolution
In this section, we conducted experiments to evaluate the impact of depthwise separable convolution on the model. As shown in Table 11, after introducing depthwise separable convolution, the computational cost decreased by 4.5 GFLOPs compared to using only standard dilated convolution, without a significant drop in accuracy. Specifically, mAP50 decreased by only 0.6%, and mAP50-95 decreased by 0.5%. This indicates that depthwise separable dilated convolution can maintain the model's accuracy while further improving its efficiency.

Discusion

Firstly, compared with AFPN, BiFPN, Unet++, U-Net, and FPN, L-FPN performs well in detection accuracy, parameter count, and computational complexity. This indicates that L-FPN can effectively extract high-resolution features and fuse them with deep semantic features, thereby improving the effectiveness of small object detection. In addition, it can be seen from the experimental results that using our lightweight feature fusion module to replace residual blocks with high computational complexity in deep layers can effectively reduce the parameter and computational complexity of the model, and improve the detection accuracy of small targets while reducing the model size and computational complexity. Secondly, as shown in Table 12, compared to AFPN and BiFPN, L-FPN has significantly faster inference speed. However, compared to the original FPN, L-FPN has a slower inference speed. This may be because L-FPN adopts high-resolution feature maps and dense

connection strategies, which may lead to frequent memory access and increase the transmission overhead of data between different layers. Moreover, L-FPN adopts a dual head asymptotic fusion strategy and a feature extraction semantic integration mechanism, which may result in the current layer's computation relying on the results of the previous layer, making it difficult to achieve high parallelism and affecting inference speed.

Overall, our model achieves a good balance between detection real-time performance, accuracy, and computational resources. This trade-off is particularly important in application scenarios such as drones, where resources are limited and both detection latency and accuracy have high demands. Although the real-time performance of L-FPN is slightly lower than that of FPN, its detection accuracy is significantly improved. Furthermore, L-FPN outperforms AFPN and BiFPN in both accuracy and real-time performance, indicating that despite fully utilizing high-resolution feature maps, our overall design maintains high computational efficiency, making it well-suited for real-time deployment on resource-constrained platforms like drones.

Conclusion

At present, most small object detection networks still rely on computationally expensive residual blocks for deep feature extraction, which, in our view, leads to considerable waste of computational resources. To address this issue, we propose a Dual-phase Asymptotic Feature Fusion mechanism (DAFF) and a Decoupled Feature Extraction-Semantic Integration mechanism (DEI), and design a feature fusion module called DSPF. Based on these designs, we construct a novel Feature Pyramid Network, named L-FPN. Furthermore, we introduce BPD-YOLO based on L-FPN. To validate its effectiveness, we conducted comparative experiments and ablation studies. The experimental results demonstrate that BPD-YOLO significantly improves small object detection accuracy from a UAV perspective and outperforms both the baseline model and several state-of-the-art methods.

Although our network has achieved remarkable improvements in both detection accuracy and computational efficiency, there is still room for further optimization in terms of lightweight design. In future work, we aim to continue exploring ways to reduce computational resource consumption and to achieve a better balance between inference speed and detection accuracy.

Data availability

The datasets used in this study, including VisDrone2019 and TinyPerson, are publicly available. he VisDrone2019 dataset can be accessed at https://github.com/VisDrone/VisDrone-Dataset,and the TinyPerson dataset is availab le at https://github.com/ucas-vg/TinyBenchmark.

Received: 24 April 2025; Accepted: 20 August 2025
Published online: 29 August 2025

References

1. Lin, T. Y. et al. Feature pyramid networks for object detection. IEEE Computer Society (2017).
2. Sun, K. et al. High-resolution representations for labeling pixels and regions. arXiv preprint arXiv:1904.04514 (2019).
3. Liu, S., Huang, D. & Wang, Y. Learning spatial fusion for single-shot object detection. arXiv preprint arXiv:1911.09516https://doi.org/10.48550/arXiv.1911.09516 (2019).
4. Xiao, Y., Xu, T., Yu, X., Fang, Y. & Li, J. A lightweight fusion strategy with enhanced interlayer feature correlation for small object detection. *IEEE Transactions on Geoscience and Remote Sensing* **62**, 1–11. https://doi.org/10.1109/TGRS.2024.3457155 (2024).
5. He, K., Zhang, X., Ren, S. & Sun, J. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Transactions on Pattern Analysis & Machine Intelligence* **37**, 1904–1916. https://doi.org/10.1109/TPAMI.2015.2389824 (2015).
6. Wang, J. et al. Deep high-resolution representation learning for visual recognition. *IEEE Transactions on Pattern Analysis & Machine Intelligence* **43**, 3349–3364. https://doi.org/10.1109/TPAMI.2020.2983686 (2021).
7. Yang, G. et al. Afpn: Asymptotic feature pyramid network for object detection. In 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2184–2189, https://doi.org/10.1109/SMC53992.2023.10394415 (2023).
8. Sun, K., Xiao, B., Liu, D. & Wang, J. Deep high-resolution representation learning for human pose estimation. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 5693–5703 (2019).
9. Ronneberger, O., Fischer, P. & Brox, T. U-net: Convolutional networks for biomedical image segmentation. In Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18, 234–241 (Springer, 2015).
10. Liu, W., Lu, H., Fu, H. & Cao, Z. Learning to upsample by learning to sample. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 6027–6037 (2023).
11. Liu, W. et al. Ssd: Single shot multibox detector. In Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14, 21–37 (Springer, 2016).
12. Lin, T. Y., Goyal, P., Girshick, R., He, K. & Dollár, P. Focal loss for dense object detection. IEEE Transactions on Pattern Analysis & Machine Intelligence **PP**, 2999–3007 (2017).
13. Redmon, J., Divvala, S., Girshick, R. & Farhadi, A. You only look once: Unified, real-time object detection. In 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788, https://doi.org/10.1109/CVPR.2016.91 (2016).
14. Zhang, Y. et al. Ffca-yolo for small object detection in remote sensing images. *IEEE Transactions on Geoscience and Remote Sensing* **62**, 1–15. https://doi.org/10.1109/TGRS.2024.3363057 (2024).
15. Zhu, X., Lyu, S., Wang, X. & Zhao, Q. Tph-yolov5: Improved yolov5 based on transformer prediction head for object detection on drone-captured scenarios. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops, 2778–2788 (2021).
16. Woo, S., Park, J., Lee, J.-Y. & Kweon, I. S. Cbam: Convolutional block attention module. In Proceedings of the European Conference on Computer Vision (ECCV), 3–19 (2018).
17. Zeng, Y. et al. Arf-yolov8: a novel real-time object detection model for uav-captured images detection. *J Real-Time Image Proc* **21**, https://doi.org/10.1007/s11554-024-01483-z (2024).
18. Zeng, S. et al. Sca-yolo: a new small object detection model for uav images. *Vis Comput* **40**, 1787–1803. https://doi.org/10.1007/s00371-023-02886-y (2024).
19. Ma, P., He, X., Chen, Y. & Liu, Y. Isod: Improved small object detection based on extended scale feature pyramid network. *The Visual Computer* **41**, 465–479 (2025).

20. Wang, P., Shi, D. & Aguilar, J. Pcp-yolo: an approach integrating non-deep feature enhancement module and polarized self-attention for small object detection of multiscale defects. *Signal, Image and Video Processing* **19**, 1–13 (2025).
21. Gao, T. et al. Msnet: Multi-scale network for object detection in remote sensing images. *Pattern Recognition* **158**, 110983. https://doi.org/10.1016/j.patcog.2024.110983 (2025).
22. Gomaa, A. & Abdalrazik, A. Novel deep learning domain adaptation approach for object detection using semi-self building dataset and modified yolov4. World Electric Vehicle Journal **15**, https://doi.org/10.3390/wevj15060255 (2024).
23. Gomaa, A. Advanced domain adaptation technique for object detection leveraging semi-automated dataset construction and enhanced yolov8. In 2024 6th Novel Intelligent and Leading Emerging Sciences Conference (NILES), 211–214, https://doi.org/10.1109/NILES63360.2024.10753164 (2024).
24. Liu, S., Qi, L., Qin, H., Shi, J. & Jia, J. Path aggregation network for instance segmentation. In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 8759–8768, https://doi.org/10.1109/CVPR.2018.00913 (2018).
25. Ghiasi, G., Lin, T.-Y. & Le, Q. V. Nas-fpn: Learning scalable feature pyramid architecture for object detection. In 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 7029–7038, https://doi.org/10.1109/CVPR.2019.00720 (2019).
26. Tan, M., Pang, R. & Le, Q. V. Efficientdet: Scalable and efficient object detection. In 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 10778–10787, https://doi.org/10.1109/CVPR42600.2020.01079 (2020).
27. Qiao, S., Chen, L.-C. & Yuille, A. Detectors: Detecting objects with recursive feature pyramid and switchable atrous convolution. In 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 10208–10219, https://doi.org/10.1109/CVPR46437.2021.01008 (2021).
28. Chen, Y. et al. Accurate leukocyte detection based on deformable-detr and multi-level feature fusion for aiding diagnosis of blood diseases. *Computers in Biology and Medicine* **170**, 107917. https://doi.org/10.1016/j.compbiomed.2024.107917 (2024).
29. Hu, J., Shen, L. & Sun, G. Squeeze-and-excitation networks. In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 7132–7141, https://doi.org/10.1109/CVPR.2018.00745 (2018).
30. Zhou, Z., Siddiquee, M. M. R., Tajbakhsh, N. & Liang, J. Unet++: Redesigning skip connections to exploit multiscale features in image segmentation. *IEEE Transactions on Medical Imaging* **39**, 1856–1867. https://doi.org/10.1109/TMI.2019.2959609 (2020).
31. Chen, Y. et al. Accurate leukocyte detection based on deformable-detr and multi-level feature fusion for aiding diagnosis of blood diseases. *Computers in Biology and Medicine* **170**, 107917. https://doi.org/10.1016/j.compbiomed.2024.107917 (2024).
32. Zhao, Q., Sheng, T., Wang, Y., Tang, Z. & Ling, H. M2det: A single-shot object detector based on multi-level feature pyramid network. *Proceedings of the AAAI Conference on Artificial Intelligence* **33**, 9259–9266 (2019).
33. Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K. & Yuille, A. L. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **40**, 834–848. https://doi.org/10.1109/TPAMI.2017.2699184 (2018).
34. Wu, X., Hong, D. & Chanussot, J. Uiu-net: U-net in u-net for infrared small object detection. *IEEE Transactions on Image Processing* **32**, 364–376. https://doi.org/10.1109/TIP.2022.3228497 (2023).
35. Lin, T. Y., Maire, M., Belongie, S., Hays, J. & Zitnick, C. L. Microsoft coco: Common objects in context. Springer International Publishing 740–755 (2014).
36. Chollet, F. Xception: Deep learning with depthwise separable convolutions. In 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1800–1807, https://doi.org/10.1109/CVPR.2017.195 (2017).
37. Everingham, M., Gool, L. V., Williams, C. K. I., Winn, J. & Zisserman, A. The pascal visual object classes (voc) challenge. *International Journal of Computer Vision* **88**, 303–338 (2010).
38. Yu, X., Gong, Y., Jiang, N., Ye, Q. & Han, Z. Scale match for tiny person detection. In 2020 IEEE Winter Conference on Applications of Computer Vision (WACV), 1246–1254, https://doi.org/10.1109/WACV45572.2020.9093394 (2020).
39. Zhu, P. et al. Detection and tracking meet drones challenge. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**, 7380–7399. https://doi.org/10.1109/TPAMI.2021.3119563 (2022).
40. Chen, Y., Lin, Z., Zhao, X., Wang, G. & Gu, Y. Deep learning-based classification of hyperspectral data. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* **7**, 2094–2107. https://doi.org/10.1109/JSTARS.2014.2329330 (2014).
41. Huang, G., Liu, Z., Van Der Maaten, L. & Weinberger, K. Q. Densely connected convolutional networks. In 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2261–2269, https://doi.org/10.1109/CVPR.2017.243 (2017).
42. Wu, Q., Li, Y., Huang, W., Chen, Q. & Wu, Y. C3tb-yolov5: integrated yolov5 with transformer for object detection in high-resolution remote sensing images. *International Journal of Remote Sensing* **45**, 2622–2650 (2024).
43. Xiong, G., Qi, J., Wang, M., Wu, C. & Sun, H. Gcge-yolo: Improved yolov5s algorithm for object detection in uav images. In 2023 42nd Chinese Control Conference (CCC), 7723–7728 (IEEE, 2023).
44. Zhang, P., Zhong, Y. & Li, X. Slimyolov3: Narrower, faster and better for real-time uav applications. In 2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW), 37–45, https://doi.org/10.1109/ICCVW.2019.00011 (2019).
45. Li, X. et al. Generalized focal loss v2: Learning reliable localization quality estimation for dense object detection. 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 11627–11636 (2020).
46. Ren, S., He, K., Girshick, R. & Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **39**, 1137–1149. https://doi.org/10.1109/TPAMI.2016.2577031 (2017).
47. Vaswani, A. et al. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17, 6000–6010 (Curran Associates Inc., Red Hook, NY, USA, 2017).
48. Redmon, J. & Farhadi, A. Yolov3: An incremental improvement. ArXiv (2018).
49. Chen, J. et al. Run, don't walk: Chasing higher flops for faster neural networks. In 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 12021–12031, https://doi.org/10.1109/CVPR52729.2023.01157 (2023).
50. Qin, D. et al. Mobilenetv4: Universal models for the mobile ecosystem. *In Computer Vision - ECCV* **2024**, 78–96 (2024).
51. Li, Y. et al. Rethinking vision transformers for mobilenet size and speed. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 16889–16900 (2023).
52. Wang, J. et al. Carafe: Content-aware reassembly of features. In 2019 IEEE/CVF International Conference on Computer Vision (ICCV), 3007–3016, https://doi.org/10.1109/ICCV.2019.00310 (2019).

Acknowledgements

This study was supported by the Key Project of Yunnan Basic Research Program (grant number 202401AS070034) and the Yunnan Provincial Forestry and Grass Science and Technology Innovation Joint Project (grant number 202404CB090002).

Author contributions

M.C. conceived the study; M.C. developed the methodology and software; C.P., Z.C., and C.Z. performed the validation; L.Y. and C.P. carried out the formal analysis; M.C. conducted the investigation and curated the data; M.C. wrote the original draft; C.P. and H.W. reviewed and edited the manuscript; H.W. provided the visualizations; Z.C. and C.Z. supervised the project; L.Y. managed the project and acquired the funding. All authors reviewed the manuscript.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to L.Y.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.